

Part 5 Section 3: Mixture Model Fitting

Aaron Robotham

Libraries needed for this chapter: magicaxis, ProFit, ProFound, data.table, mixtools

```
library(magicaxis, quietly=TRUE)
library(ProFit, quietly=TRUE)
library(ProFound, quietly=TRUE)
library(data.table, quietly=TRUE)
library(mixtools, quietly=TRUE)
library(Rfits, quietly=TRUE)
library(rgl, quietly=TRUE)
options(rgl.useNULL = TRUE)
```

So everybody running this chapter gets the same outputs, we should first set the seed.

```
set.seed(1)
```

1D Mixture Modelling

Mixture modelling is very common in physics and astronomy. Say, for example, you are looking at the flux output (so number of photons observed) of a spectrograph around a doublet feature. It might look a bit like this:

```
spec1d = c(rnorm(100,mean=6563,sd=0.5), rnorm(50,mean=6565,sd=1))
maghist(spec1d, xlab='Wavelength / Ang', ylab='Photons', breaks=20)
```

```
## Summary of used sample:
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      6562   6563   6563   6564   6564   6567
```

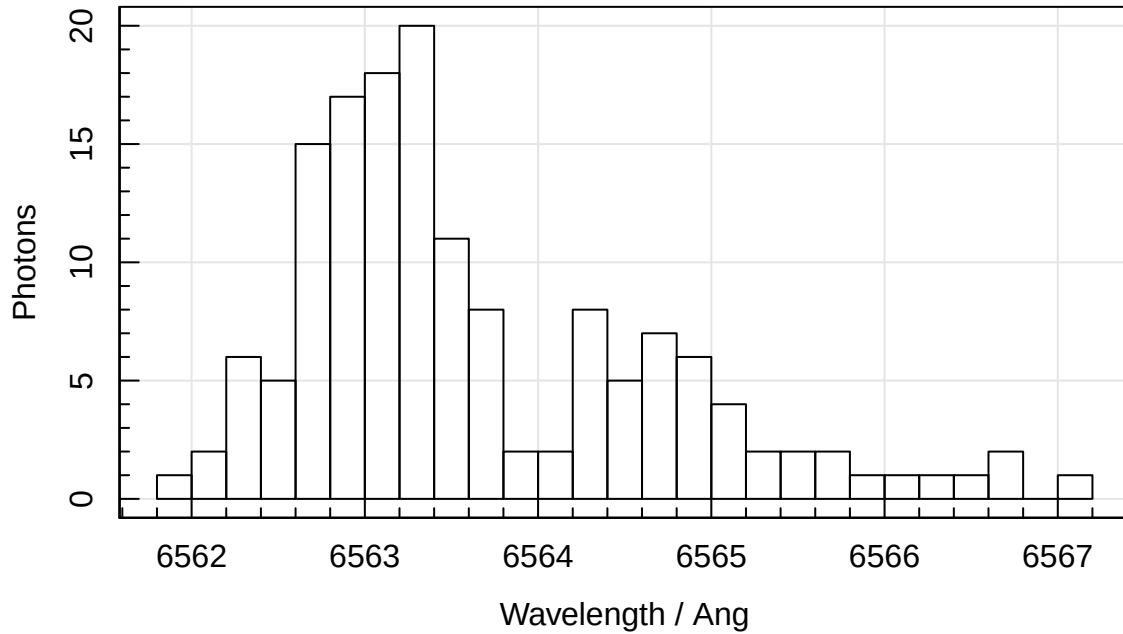
```
## Pop Std Dev: 1.0581
```

```
## MAD: 0.79622
```

```
## Half 16-84 Quan (1s): 1.032
```

```
## Half 02-98 Quan (2s): 2.0745
```

```
## Using 150 out of 150
```



Decomposing the above into the generative Normal distributions that created the data is what we mean by mixture modelling, and it is a type of analysis that can only be properly solved with a Bayesian approach. First we can write down the likelihood function we wish to maximise:

$$\ln L = \sum_{i=1}^N \ln(\lambda \mathcal{N}(x_i, \mu_1, \sigma_1) + (1 - \lambda) \mathcal{N}(x_i, \mu_2, \sigma_2))$$

where x_i is observation i of N , μ_1 is mean 1, μ_2 is mean 2, σ_1 is standard deviation 1, σ_2 is standard deviation 2, and λ is the mixture term which controls what fraction of the PDF is provided by model 1. Next we can encode this computationally:

```
mix_like = function(parm, Data){
  mix1 = dnorm(Data, mean=parm[1], sd=parm[2]) * parm[5]
  mix2 = dnorm(Data, mean=parm[3], sd=parm[4]) * (1 - parm[5])
  like = mix1 + mix2
  return(-sum(log(like)))
}
```

We can optimise this using BFGS with sensible limits:

```
out1d = optim(par=c(6560,2,6570,2,0.5), Data=spec1d, fn=mix_like, method = 'L-BFGS-B',
  lower=c(6550,0.1,6550,0.1,0.1), upper=c(6600,1.5,6600,1.5,0.9))
out1d$par
```

```
## [1] 6564.5515545    1.0477642 6563.0244458    0.4042074    0.4110206
```

This is a pretty good result- we get the means and standard deviations almost exactly correct, and the mixing term (5th parameter that determine the fraction of photons in each feature) is also close to 1/3, which is what we expect). Note something interesting though, the mixture positions have switched from our initial guess ordering, i.e. the larger mean is parameter 1 not parameter 3. Such switching is a very common feature (problem) in mixture modelling, and you need to be careful when processing the results that you handle such switching correctly.

Not surprisingly, there are a number of fully featured packages that tackle this class of problem trivially in **R**. The best of the lot is **mixtools**, where we can just specify the number of mixtures to find ($k = 2$ in this case):

```
mix1d = normalmixEM(spec1d, k=2)
```

```
## number of iterations= 92
```

```

mix1d$mu
## [1] 6563.024 6564.552
mix1d$sigma
## [1] 0.4042131 1.0477217
mix1d$lambda
## [1] 0.5889932 0.4110068

```

It has basically converged on the same solution, but with much less effort on our part!

2D Profile Modelling

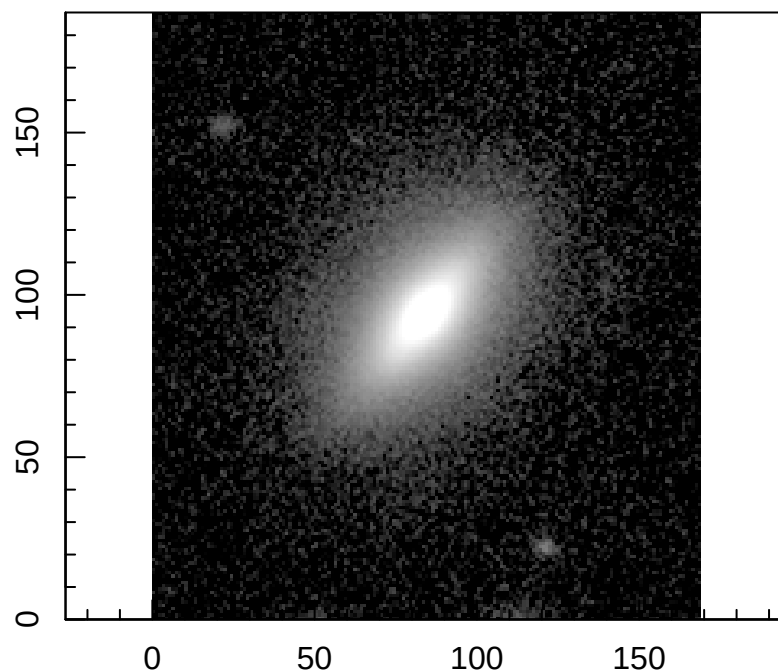
A powerful application of the Bayesian approach is the 2D mixture modelling of galaxy light profiles

First let us take a look at some real data from KiDS (a survey on the VST optical telescope):

```
image = Rfits_read_image(system.file("extdata", "KiDS/G265911fitim.fits", package="ProFit"))$imDat
```

We can plot the data easily:

```
magimage(image)
```



It is interesting to see this in 3D:

```

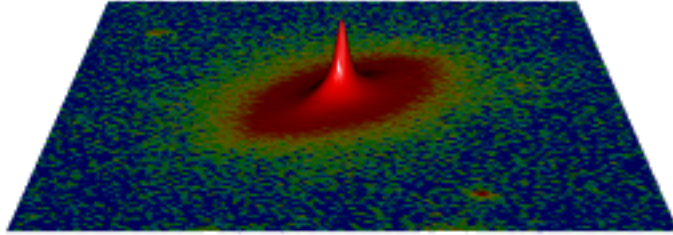
surface3d(x=1:169, y=1:187, z=image*1e10, col=hsv(magmap(image,stretch='log',locut=0.1,
                                                           hicut=0.9,bad=2/3,flip=TRUE)$map))
rglwidget()

```

```

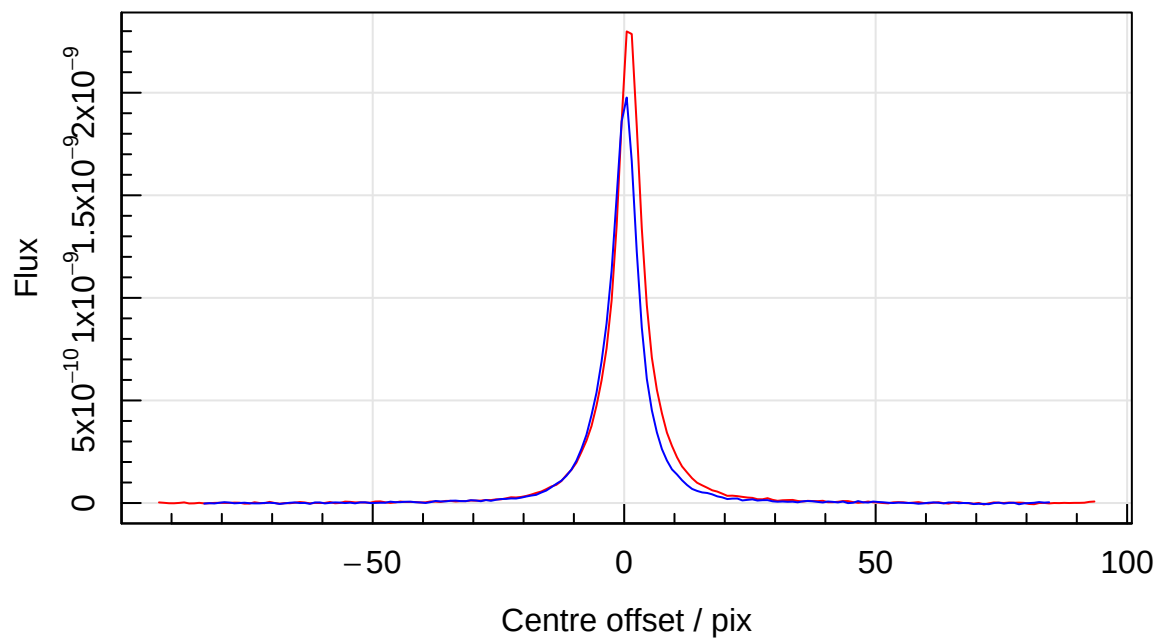
## Warning in snapshot3d(scene = x, width = width, height = height): webshot =
## TRUE requires the webshot2 package and Chrome browser; using rgl.snapshot()
## instead

```

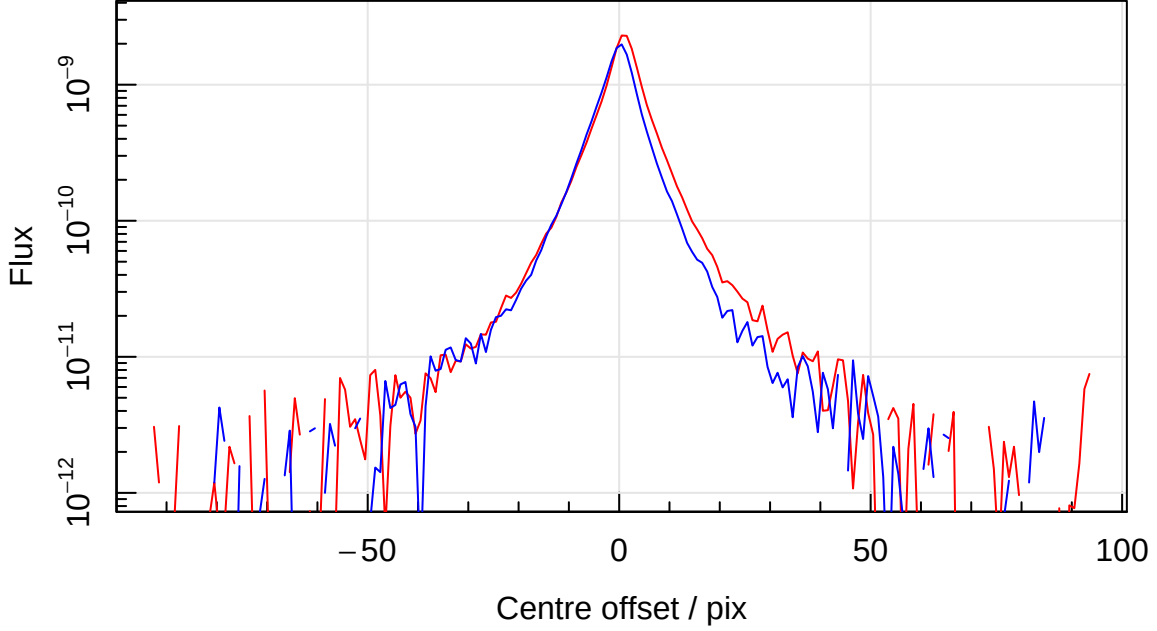


Even the un-keen observer will notice a few things- the flux is largest (image whitest/reddest) in the centre, and from there it drops off fairly monotonically until it smoothly disappears into the noisy background. We can show this more clearly by making image slices through the data (shown here in linear and log space):

```
magplot(1:187-187/2,image[169/2,], type='l', col='red', xlab='Centre offset / pix', ylab='Flux')
lines(1:169-169/2,image[,187/2], col='blue')
```



```
magplot(1:187-187/2,image[169/2,], type='l', col='red',log='y', xlab='Centre offset / pix', ylab='Flux')
lines(1:169-169/2,image[,187/2], col='blue')
```



Here we can note a few things. The fact that there is still curvature in the profile even in log space means the galaxy itself must have a *steeper* than linear (which in actual linear space means exponential) flux profile. We also see some smoothing of the profile in the centre. This effect is not intrinsic, and is in fact due atmospheric blurring (or *seeing*) convolving the image data with a kernel and smoothing it out on a characteristic scale.

If we want to physically interpret this type of data, we need to be able to generate a model with the appropriate intrinsic flux profile and spatial shape which is also blurred by a realistic model of the atmospheric seeing. This model need to be constrained against the image data, where in some sense we want to produce the model that best represents the data. Hopefully by now this is sounding like a prime candidate for Bayesian modelling!

In order to deal with this sort of problem I developed (along with some key collaborators, see the paper co-authors) the **R** package **ProFit** (Robotham) that provides the flexibility and framework to robustly decompose galaxies into their constituent components using a rigorous Bayesian modelling approach. In this chapter we will look at the practicalities and implementation of this framework which is now being used to profile 100,000s of galaxies across a number of surveys.

The Sersic Profile

For galaxy fitting purposes the Sersic profile (Sersic, 1963) is a modern day work-horse, having enough flexibility in profile characteristics to fit a large variety of galaxy types. The flux profile is usually parametrised as:

$$I(R) = I_e \exp \left\{ -b_n \left[\left(\frac{R_m}{R_e} \right)^{1/n} - 1 \right] \right\},$$

where I_e is the intensity of the profile at R_e (the radius containing half of the flux), n is the Sersic index (which has special cases of Normal / Gaussian with $n = 0.5$, exponential with $n = 1$ and de-Vaucouleurs with $n = 4$) and b_n is a derived parameter that ensures the correct integration properties at R_e and is the quantile at which the Gamma probability distribution function integrates to 0.5 given a shape parameter of $2n$. This can be computed directly using high-level interfaces to statical distribution libraries (e.g. in **R** `b_n = qgamma(0.5, 2*n)`). R_m is the modified radius where we want to make the evaluation, which in one dimension is simply the radius from the profile centre. In two dimensions R_m has a more complex form

$$R_m = \sqrt{(x - x_{cen})^2 + (y - y_{cen})^2}$$

where x, y is the two-dimensional location where we wish to make the evaluation and x_{cen}, y_{cen} is the location of the profile centre. When the two-dimensional profile is non-circular, the isophotal contours become elliptical annuli, so calculating R_m becomes a series of computations that rotate and circularize the annuli. When the major-axis angle θ is defined such that it is 0° vertically and increases positively as the galaxy is rotated counter-clockwise (as per GalFit) these steps become

$$R_t = \sqrt{(x - x_{cen})^2 + (y - y_{cen})^2}, \theta_t = \arctan \frac{x - x_{cen}}{y - y_{cen}} + \theta, R_m = \sqrt{(R_t \sin(\theta_t)/A_{rat})^2 + (R_t \cos(\theta_t))^2},$$

where A_{rat} is the minor to major axis ratio (so always a number between 0 and 1, where 0 is an infinitely thin line and 1 is a circle or disc). Note this form is slightly different to the original **ProFit** paper because there is a typo in the type-setting of the paper (AGGGHHH).

The final modifier to the standard one-dimensional Sersic profile intensity is to allow for apparent ‘boxiness’. Boxiness is often used to create the more rectangular visual appearance of galaxy bulges or elliptical galaxies. It effectively modifies the unit circle (or ellipse) away from the standard L^2 norm and either towards a diamond/kite appearance (L^1 norm) or a boxy appearance (L^∞ norm). This is achieved though a small manipulation of the final R_m calculation above

$$R_m = \left[(R_t \sin(\theta_t) A)^{(2+B)} + (R_t \cos(\theta_t))^{(2+B)} \right]^{\frac{1}{2+B}},$$

where B is the boxiness, which for practical (as opposed to mathematical) purposes is usually allowed to vary between -1 and 1 (thus covering the full range of apparent visual boxiness).

It is not necessary to parametrise the profile using I_e to define the normalization, as this is rarely intuitive. Instead, the user can specify the total flux contained within a profile in magnitudes (m) given a magnitude zero-point (m_{zero}), and calculations are made internally to convert this to the correct value of I_e . These include appropriate modifications for the axis ratio (A_{rat}) and boxiness (B):

$$R_{box} = \frac{\pi(2+B)}{2\beta\left(\frac{1}{2+B}, \frac{1}{2+B}\right)}, L_{tot} = \frac{2\pi A_{rat} R_e^2 n \Gamma(2n) \exp(b_n)}{R_{box} b_n^{2n}}, I_e = \frac{10^{-0.4(m-m_{zero})}}{L_{tot}},$$

where β is the Beta function defined as $\beta(a, b) = \Gamma(a)\Gamma(b)/\Gamma(a+b)$, and Γ is the standard Gamma function. If the profile has no boxiness, then the above is simplified by noting that $R_{box}(B=0) = 1$ and therefore the denominator term for L_{tot} disappears.

The above means that to reasonably describe a galaxy profile we might need 7-8 parameters: x-centre (x_{cen}), y-centre (y_{cen}), magnitude (m), effective radius (R_e), Sersic index (n), rotation angle (θ_t), axial ratio (A_{rat}) and boxiness (B), where we might ignore boxiness in certain situations. Since each and every source we wish to fit will need this many parameters (if building a mixture model of Sersic profiles) the number of parameters we need to marginalise over can become very large very quickly.

Data-Model Likelihood

A number of inputs are required to meaningfully fit a profile to a target galaxy observation. For good-quality data where the whole image is to be used for fitting, the bare minimum is strictly an image where pixels contain integer photo-electron counts and an analytical description of the PSF.

In practice, a more complete set of inputs can include the flux image (D , in whatever linear units are appropriate), a segmentation map (that is pixel matched to D , where distinct integer values represent different detected structures in the image as per SExtractor), a binary mask (that is pixel matched to D , where TRUE means mask out and ignore for analysis, and FALSE means do not mask), and the σ image (σ , that is pixel matched to D , where values represent Normal errors in the same units as for D).

Assuming we have such a σ map of per pixel errors, and we are operating in the Normal regime (where each pixel has counted a large number of photons), the data-model likelihood can be computed with:

$$x_{i,j} = \frac{(D_{i,j} - M_{i,j})^2}{2\sigma_{i,j}^2}, \ln \mathcal{L} = \frac{\ln 2\pi}{2} - \sum_{i=1}^{N_i} \sum_{j=1}^{N_j} x_{i,j} - \ln \sigma_{i,j}.$$

Here, $D_{i,j}$ represents the observed pixel data at pixel i, j , $M_{i,j}$ represents a computed **ProFit** model at pixel i, j and $\sigma_{i,j}$ represents the estimated Normal uncertainty at pixel i, j . For a typical profile this sum is done over 100s or 1,000s of pixels, so we are rarely S/N limited when trying to fit our model.

ProFit also offer routes to compute χ^2 , Poisson and Student-t distribution likelihoods, which become appropriate for certain classes of data.

Model Specification

ProFit offers a compact method to fully describe the galaxy model using a nested list structure. A number of recognised profile types are allowed, and for each of these the necessary parameters are described as vectors (where the length is the number of profiles to be fitted), e.g.:

```
modellist1 = list(
  sersic = list(
    xcen = c(160, 60),
    ycen = c(120, 50),
    mag = c(15, 13),
    re = c(14, 5),
    nser = c(3, 10),
    ang = c(46, 80),
    axrat = c(0.4, 0.6),
    box = c(0.5, -0.5)
  ),

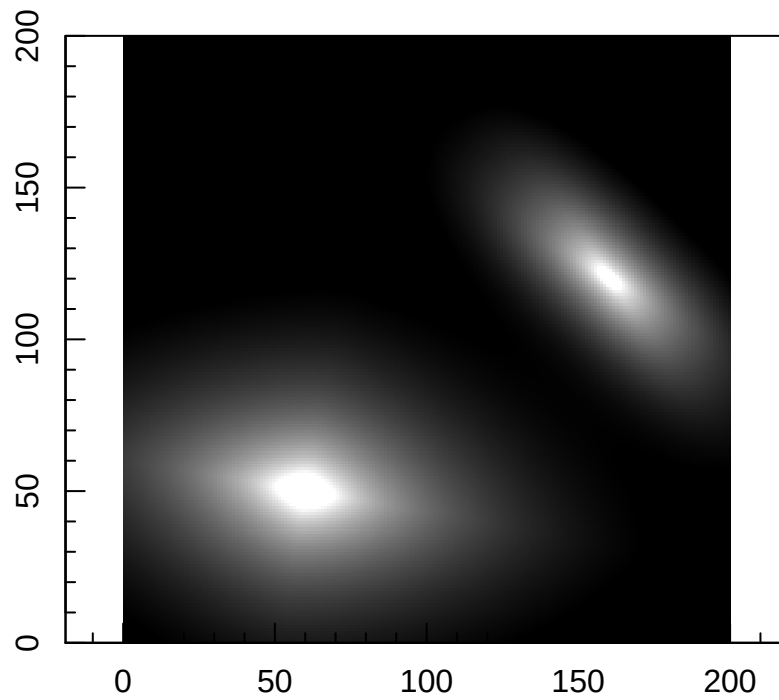
  pointsource = list(
    xcen = c(34, 10, 150),
    ycen = c(74, 120, 30),
    mag = c(10, 13, 16)
  ),

  sky = list(
    bg = 3e-12
  )
)
```

Here we have 2 Sersic profiles, 3 point sources (usually used for stars) and the sky level. We can then use a helper function **profitMakeModel** to create the target model image.

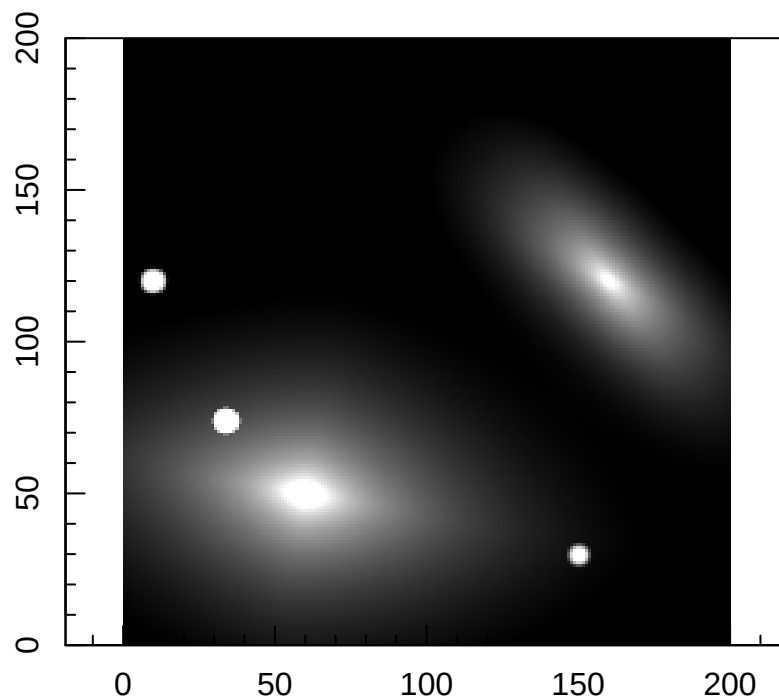
Without a PSF provided only the extended sources are shown, with no convolution:

```
magimage(profitMakeModel(modellist=modellist1, dim=c(200,200)))
```



With a PSF provided the PSFs are displayed and the extended sources are convolved with the PSF:

```
magimage(profitMakeModel(modellist=modellist1, psf=profitMakePointSource(), dim=c(200,200)))
```



Making Mock Data

Here we are going to create and then fit a simple example with **ProFit**, highlighting the main concepts.

```
modellist2 = list(
  sersic = list(
    xcen = 50,
    ycen = 50,
    mag = 15,
```



```

    re = 5,
    nser = 2,
    ang = 135,
    axrat = 0.5
),
sky = list(
  bg = 1600
)
)

mock_image = profitMakeModel(modellist=modellist2, dim=c(100,100), magzero=30)$z

```

Here *mock_image* is being created to be in photon counts. Given we are in the Normal regime, we can add noise and make a ‘realistic’ mock image. We add a background sky level of 1600 counts to all pixels to represent the noisy sky background which is not described by our simple model:

```

mock_error = sqrt(mock_image)
mock_image_noise = matrix(rpois(1e4, lambda=mock_image), nrow=100)

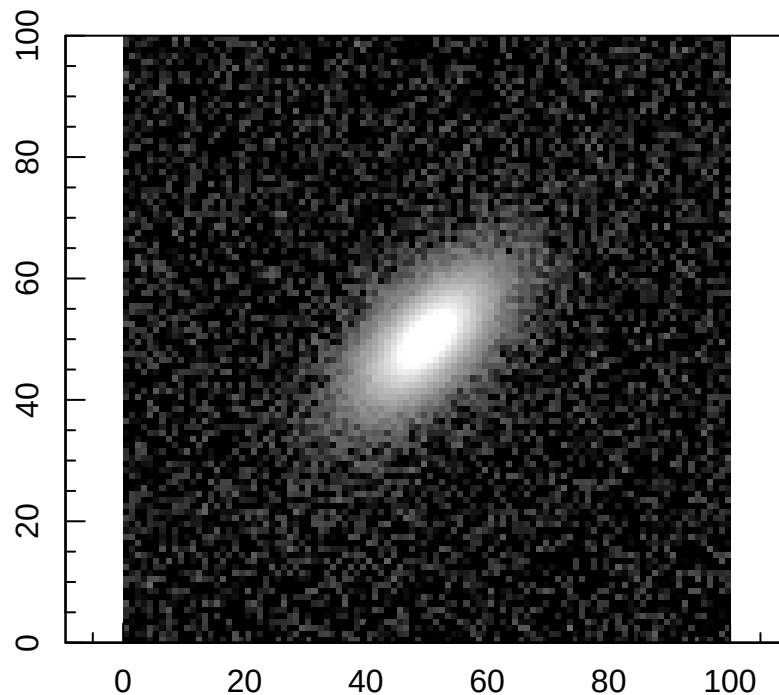
```

This noisy image should look a little bit like the real galaxy example we saw earlier:

```

magimage(mock_image_noise - 1600) #We take off the sky to get a nice visual scaling

```



Fitting Mock Data

Copying the shape of the data list structure from above, we can now specify which parameters we wish to fit:

```

tofit2 = list(
  sersic = list(
    xcen = TRUE,
    ycen = TRUE,
    mag = TRUE,
    re = TRUE,
    nser = TRUE,
    ang = TRUE,
    axrat = TRUE
  )
)

```

```

),
sky = list(
  bg = FALSE
)
)

```

Some parameters (those which represent arbitrary scales) are better fit in log space, which we can specify in a similar manner:

```

tolog2 = list(
  sersic = list(
    xcen = FALSE,
    ycen = FALSE,
    mag = FALSE, #This is already logged
    re = TRUE,
    nser = TRUE,
    ang = FALSE,
    axrat = TRUE
  ),
  sky = list(
    bg = FALSE
  )
)

```

Usually it is sensible to provide extreme limits for the parameters (even if priors are being used), since it stops the optimiser trying to make physical or numerically impossible models:

```

intervals2 = list(
  sersic = list(
    xcen = list(lim=c(0,100)),
    ycen = list(lim=c(0,100)),
    mag = list(lim=c(10,30)),
    re = list(lim=c(1,100)),
    nser = list(lim=c(0.5,20)),
    ang = list(lim=c(-180,360)),
    axrat = list(lim=c(0.1,1))
  ),
  sky = list(
    bg = c(0,Inf)
  )
)

```

We then define the data structure with a useful helper function. Here we also define the model likelihood type to be 'pois', i.e. Poisson. Other options are Normal ('norm'), Student-t ('t'), and Chi-Square ('chisq'). The keen student should attempt to make and fit similar mock data using these other distributions.

```

Data = profitSetupData(image=mock_image_noise, sigma=mock_error, modellist=modellist2,
  tofit=tofit2, tolog=tolog2, intervals=intervals2, magzero=30,
  algo.func='optim', like.func='pois')

```

Now we can compute the LP of our input model against this toy data and check the residuals:

```

(profitLikeModel(parm=Data$init, Data=Data, makeplots=TRUE))

```

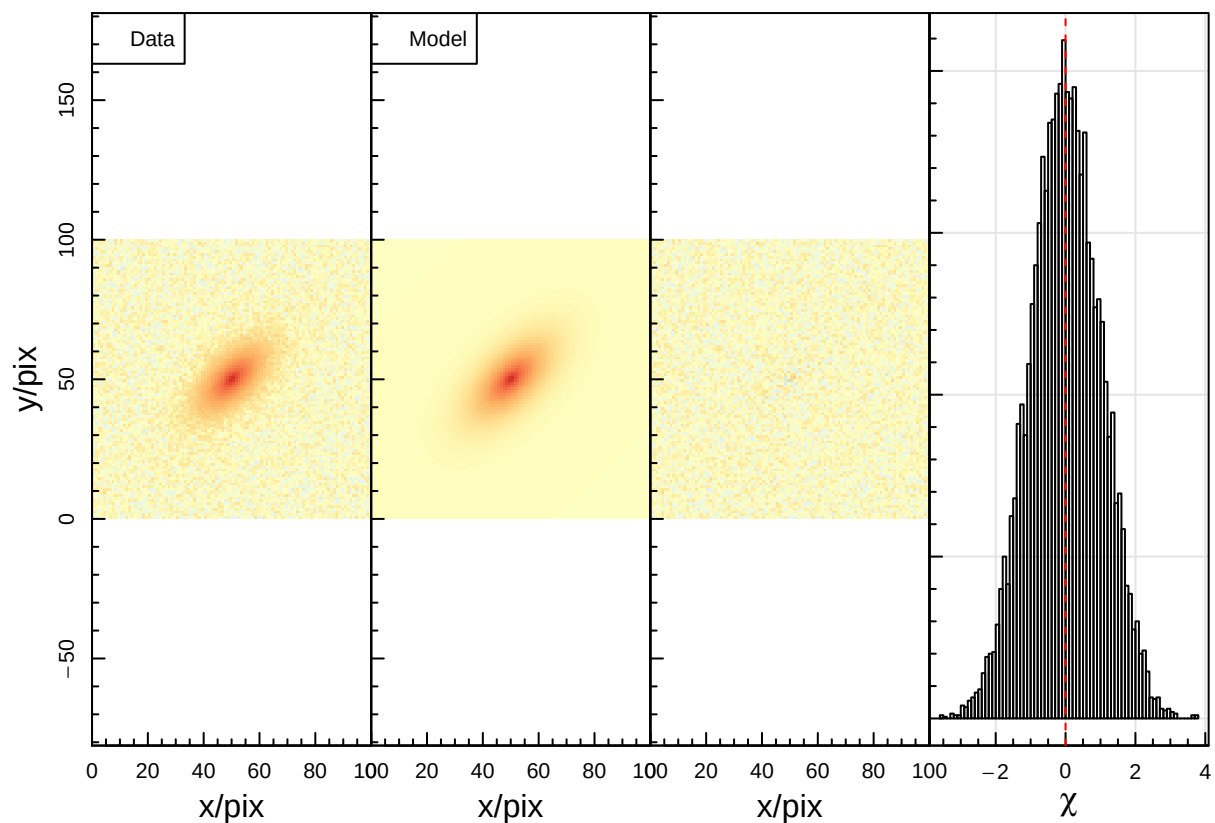
```
## Summary of used sample:
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## -3.548264 -0.672583 -0.004428  0.005555  0.684195  3.800000
```

```
## Pop Std Dev: 1.0067
```

```
## MAD: 1.0038
```

```
## Half 16-84 Quan (1s): 1.0156
## Half 02-98 Quan (2s): 2.0119
## Using 10000 out of 10000
```



```
## [1] 110436120
```

Finally we will run the **R** `optim` function much like we did earlier in the course:

```
optimfit = optim(Data$init, profitLikeModel, method='BFGS', Data=Data,
                 control=list(fnscale=-1), hessian=TRUE)
```

We can then check the fit and the fit errors:

```
optimfit$par
```

```
## sersic.xcen sersic.ycen sersic.mag sersic.re sersic.nser sersic.ang
## 50.0003651 50.0001435 14.9997756 0.6994659 0.3008171 134.9999921
## sersic.axrat
## -0.3006979
```

```
sqrt(abs(diag(solve(optimfit$hessian))))
```

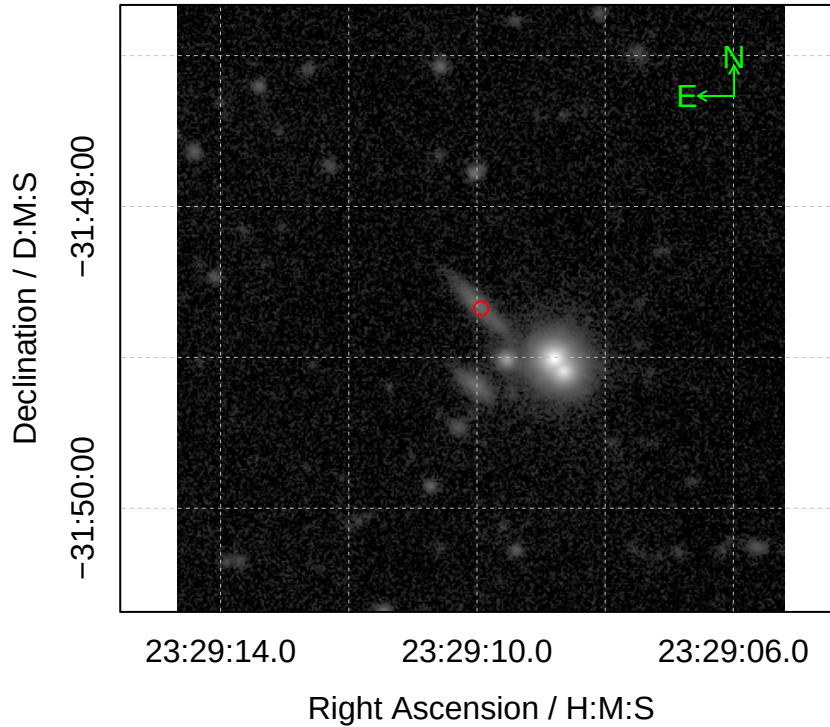
```
## sersic.xcen sersic.ycen sersic.mag sersic.re sersic.nser sersic.ang
## 0.0069136718 0.0145985119 0.0020352860 0.0011950055 0.0017024230 0.4951830218
## sersic.axrat
## 0.0009441893
```

In every case the modal fit is within the true parameter values given the estimated covariance matrix calculated via the Hessian.

Running on Real Data

A key part to efficiently running **ProFit** is identifying which pixels need fitting, and having reasonable initial guesses for the various parameters we want to fit (meaning we do not have a long MCMC burn-in phase, which can be costly). To help with this, **ProFit** has a partner package called **ProFound** (Robotham 2018) which automates a lot of these “data preparation” tasks. **ProFound** itself comes bundled with some test data that we will use here. First we will load it and take a look:

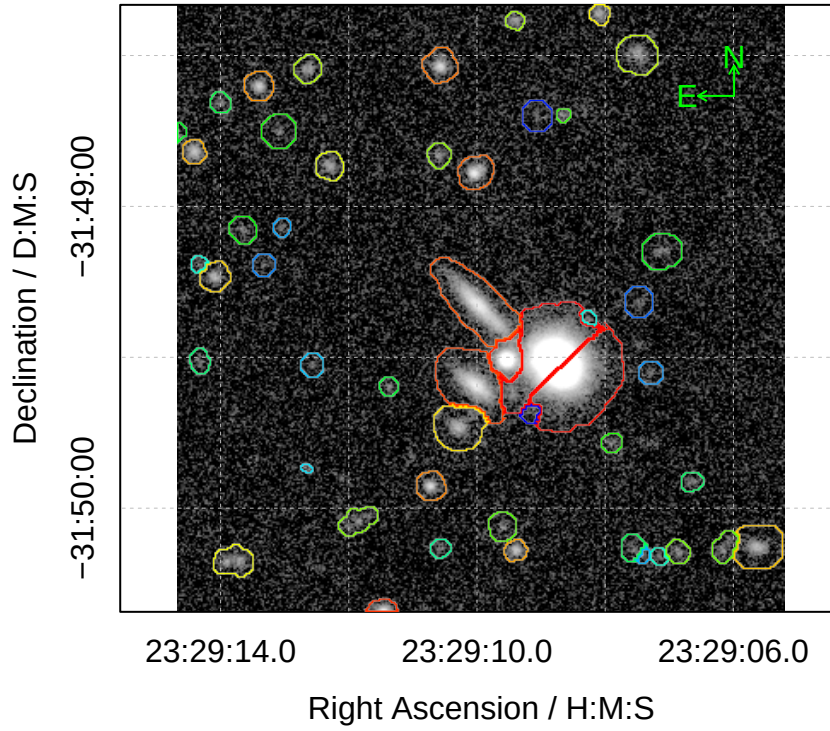
```
image = Rfits_read_image(system.file("extdata", 'VIKING/mystery_VIKING_Z.fits', package="ProFound"))  
plot(image, hicut=1)  
points(178,178,col='red')
```



Segmentation Map

The galaxy we want to fit in this example is highlighted in red in the centre. We will run **ProFound** on this image using default arguments:

```
profound = profoundProFound(image, plot=TRUE, magzero=30)
```

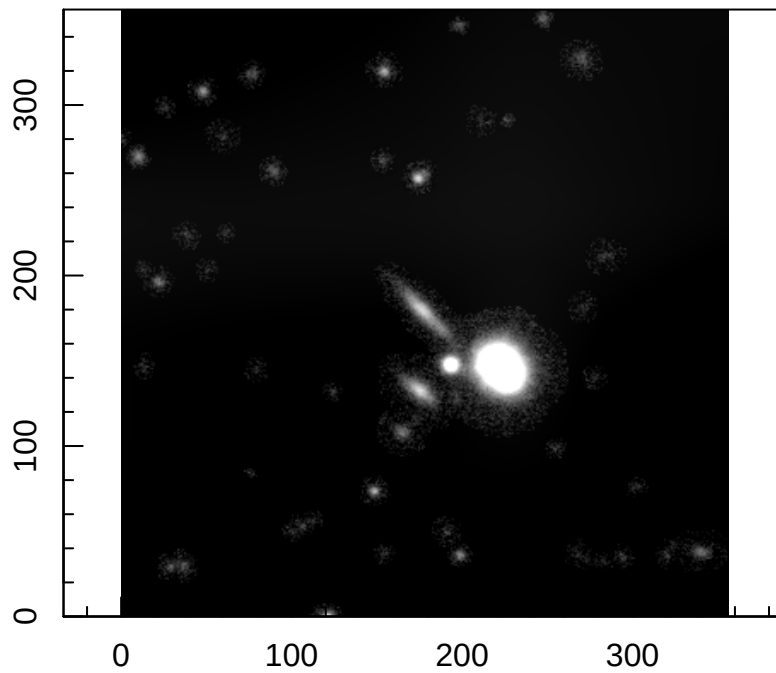


In the above image we see the **ProFound** segmentation map, which is its best guess as to which pixels belong to which sources. Visually this seems to have done a pretty good job, with the nearby bright sources well separated from the galaxy we want to fit. To make later steps easier, we will extract the source catalogue, and convert it to a *data.table*:

```
segstats = data.table(profound$segstats)
```

Using the outputs from **ProFound** we can also make a sigma map, which reflects our uncertainty per pixel:

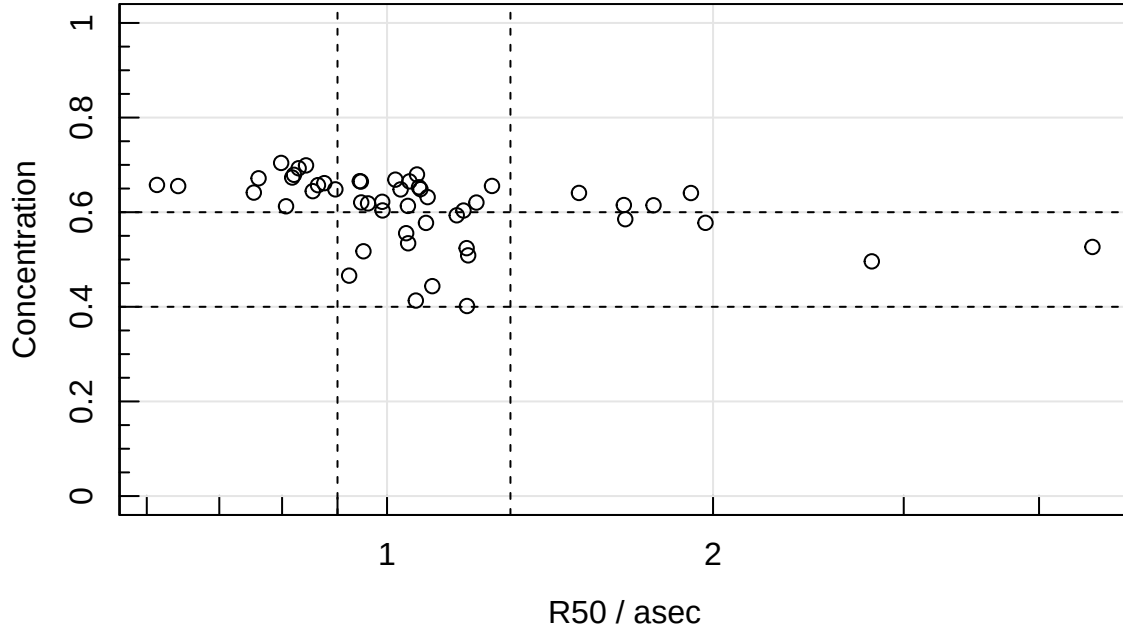
```
sigma = profoundMakeSigma(profound$image, objects=profound$objects, gain=1, sky=profound$sky, skyRMS
```



Point Spread Function (PSF)

Next we want to find the PSF for fitting, so we should extract a a bright but non-saturated star from the list above:

```
magplot(segstats[,list(R50,con)],log='x',ylim=c(0,1), xlab='R50 / asec', ylab='Concentration')
abline(h=c(0.4,0.6), lty=2)
abline(v=c(0.9,1.3), lty=2)
```



```
starlist = segstats[axrat>0.9 & con>0.4 & con<0.6 & R50>0.9 & R50<1.3,]
print(starlist)
```

##	segID	uniqueID	xcen	ycen	xmax	ymax	RAcen	Deccen	RAmax
##	<int>	<num>	<num>	<num>	<num>	<num>	<num>	<num>	<num>
## 1:	8	155320	154.08790	319.81892	154.5	319.5	352.2940	-31.80894	352.2940
## 2:	9	149074	148.53576	73.77021	148.5	73.5	352.2947	-31.83211	352.2947
## 3:	10	48308	47.91953	307.72107	47.5	307.5	352.3058	-31.81008	352.3058
## 4:	12	11270	10.08392	269.43808	10.5	269.5	352.3100	-31.81369	352.3099
## 5:	14	23196	22.15588	196.26323	22.5	195.5	352.3087	-31.82058	352.3086
##	Decmax	sep	flux	mag	flux_app	mag_app	cenfrac	N50	
##	<num>	<num>	<num>	<num>	<num>	<num>	<num>	<num>	
## 1:	-31.80897	0.17664927	6189.088	20.52093		0	NA	0.02607793	36.12613
## 2:	-31.83214	0.09240052	4443.128	20.88078		0	NA	0.03871503	24.10122
## 3:	-31.81010	0.16075739	4128.262	20.96058		0	NA	0.03105125	27.09978
## 4:	-31.81368	0.14260538	4346.299	20.90470		0	NA	0.02648044	33.66373
## 5:	-31.82065	0.28381773	3197.106	21.23811		0	NA	0.03097002	29.81087
##	N90	N100	R50	R90	R100	SB_N50	SB_N90	SB_N100	
##	<num>	<int>	<num>	<num>	<num>	<num>	<num>	<num>	
## 1:	139.57809	364	1.1882659	2.335670	3.771843	22.81906	23.64837	24.57469	
## 2:	90.03224	271	0.9510244	1.838109	3.189017	22.73945	23.53217	24.61420	
## 3:	94.92620	276	1.0458844	1.957464	3.337761	22.94657	23.66944	24.71385	
## 4:	95.59266	187	1.1597527	1.954324	2.733412	23.12618	23.62115	24.23530	
## 5:	89.42453	277	1.0863441	1.881518	3.311463	23.32762	23.88214	24.99531	
##	xsd	ysd	covxy	corxy	con	asymm	flux_reflect		
##	<num>	<num>	<num>	<num>	<num>	<lgcl>	<num>		
## 1:	3.672123	3.615042	-0.8584764	-0.06466924	0.5087473	NA	NA		
## 2:	3.048265	3.006684	0.1999036	0.02181123	0.5173929	NA	NA		
## 3:	3.209157	3.106886	-0.9326870	-0.09354482	0.5343058	NA	NA		
## 4:	2.669031	2.873008	-0.3851877	-0.05023218	0.5934291	NA	NA		

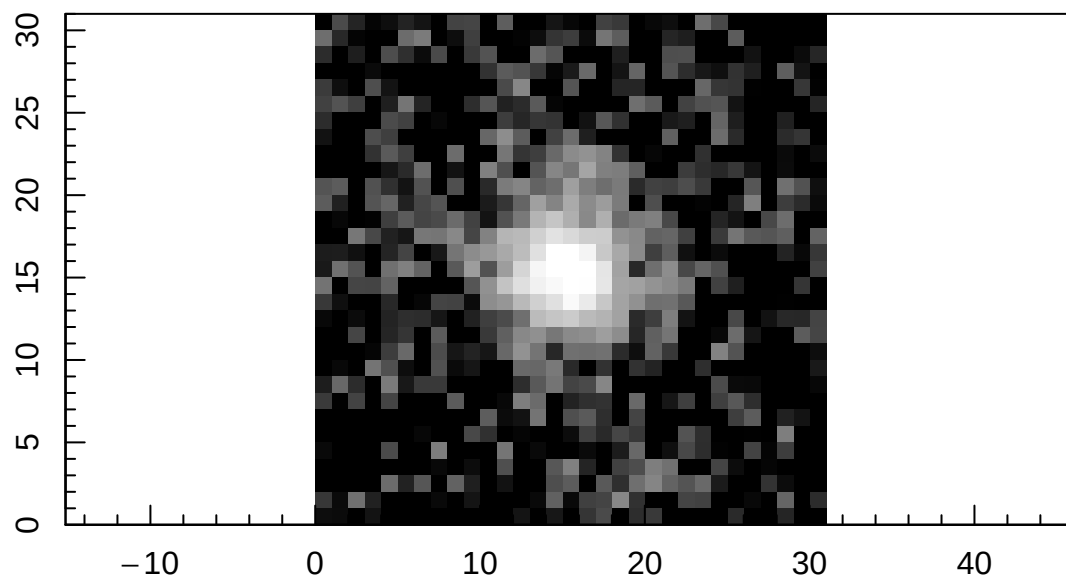
```
## 5: 3.325134 3.244729 0.8157342 0.07560683 0.5773764 NA NA
## mag_reflect semimaj semimin axrat ang signif FPlim flux_err
## <lgcl> <num> <num> <num> <num> <num> <num> <num>
## 1: NA 3.774010 3.532214 0.9359311 51.80920 Inf 2.762035 170.3861
## 2: NA 3.079871 3.002187 0.9747770 118.90014 Inf 2.857004 145.2766
## 3: NA 3.323563 3.011980 0.9062502 54.55007 Inf 2.851196 146.5277
## 4: NA 2.907968 2.662384 0.9155481 17.13681 Inf 2.972795 123.5074
## 5: NA 3.425369 3.165169 0.9240374 126.02934 30.22906 2.850046 151.5293
## mag_err flux_err_sky flux_err_skyRMS flux_err_shot flux_err_cor cor_seg
## <num> <num> <num> <num> <num> <num>
## 1: 0.02989040 14.21650 169.7919 0 0 0
## 2: 0.03550022 14.99053 144.5011 0 0 0
## 3: 0.03853691 13.99892 145.8575 0 0 0
## 4: 0.03085303 12.79143 122.8432 0 0 0
## 5: 0.05145933 19.64645 150.2503 0 0 0
## sky_mean sky_sum skyRMS_mean Nedge Nsky Nobject Nborder Nmask
## <num> <num> <num> <lgcl> <lgcl> <lgcl> <lgcl> <lgcl>
## 1: -0.69339103 -252.39434 8.899496 NA NA NA NA NA
## 2: 0.06278522 17.01479 8.777816 NA NA NA NA NA
## 3: -0.29466196 -81.32670 8.779545 NA NA NA NA NA
## 4: 0.45498014 85.08129 8.983148 NA NA NA NA NA
## 5: 0.58228245 161.29224 9.027625 NA NA NA NA NA
## edge_frac edge_excess flag_border iter origfrac Norig skyseg_mean
## <lgcl> <lgcl> <lgcl> <num> <num> <int> <lgcl>
## 1: NA NA NA 2 0.8264710 122 NA
## 2: NA NA NA 2 0.8322204 77 NA
## 3: NA NA NA 2 0.8207472 80 NA
## 4: NA NA NA 1 0.8773458 97 NA
## 5: NA NA NA 2 0.8020008 79 NA
## flag_keep
## <lgcl>
## 1: TRUE
## 2: TRUE
## 3: TRUE
## 4: TRUE
## 5: TRUE
```

We have trimmed down our list to 5 objects that look like stars. We can extract the brightest of these easily:

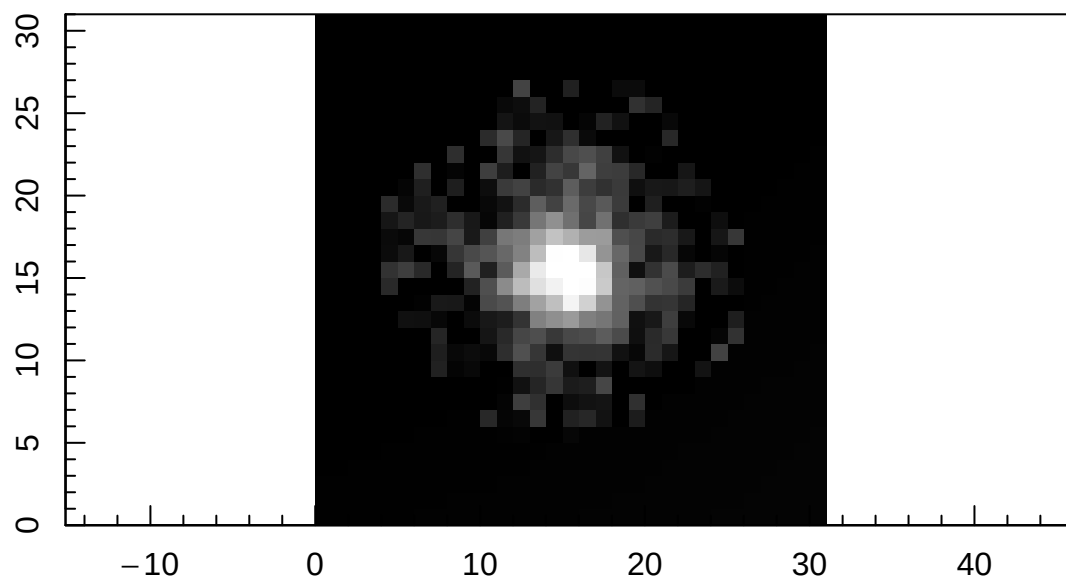
```
psf_image = magcutout(profound$image, loc=starlist[1,list(xcen,ycen)], box=c(31,31))
psf_sigma = magcutout(sigma, loc=starlist[1,list(xcen,ycen)], box=c(31,31))
psf_segim = magcutout(profound$segim, loc=starlist[1,list(xcen,ycen)], box=c(31,31))
```

We can check what it looks like:

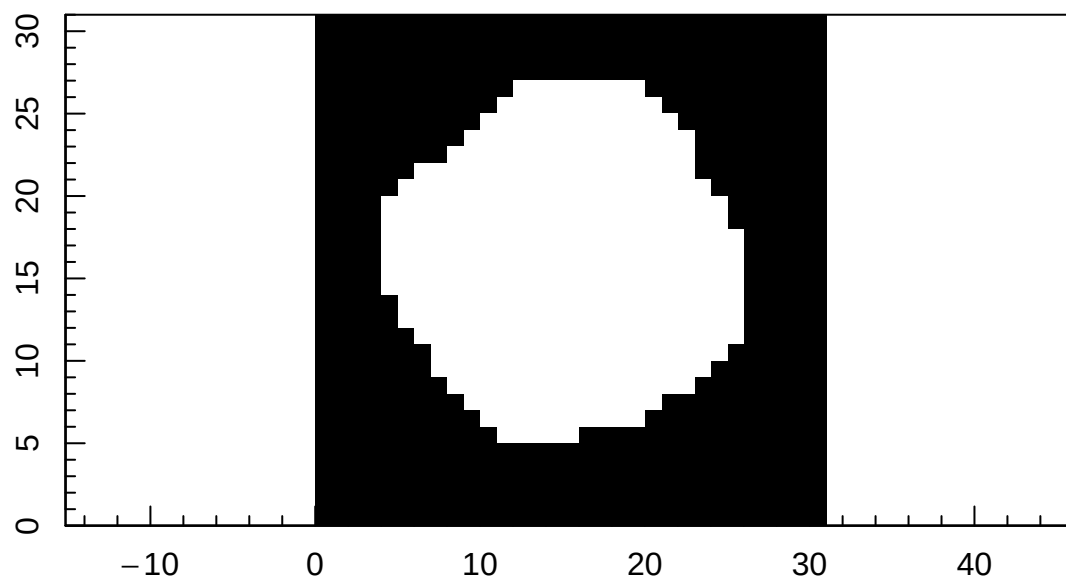
```
magimage(psf_image$image)
```



```
magimage(psf_sigma$image)
```



```
magimage(psf_segim$image)
```



Next we make our initial model, where we take our PSF estimates from the segmentation statistics:

```
psf_x = as.numeric(psf_image$loc[1])
psf_y = as.numeric(psf_image$loc[2])
psf_mag = as.numeric(starlist[1,'mag'])
psf_fwhm = as.numeric(starlist[1,'R50']*2/0.339)
psf_con = as.numeric(1/starlist[1,'con'])

psf_modellist = list(
  moffat = list(
    xcen = psf_x,
    ycen = psf_y,
    mag = psf_mag,
    fwhm = psf_fwhm,
    con = psf_con,
    axrat = 1,
    box = 0
  )
)
```

We also need to set up the *tofit*, *tolog* and *intervals* structures:

```
psf_tofit = list(
  moffat = list(
    xcen = TRUE,
    ycen = TRUE,
    mag = TRUE,
    fwhm = TRUE,
    con = TRUE,
    axrat = FALSE,
    box = FALSE
  )
)

psf_tolog = list(
  moffat = list(
    xcen = FALSE,
    ycen = FALSE,
    mag = FALSE,
    fwhm = TRUE,
    con = TRUE,
    axrat = FALSE,
    box = FALSE
  )
)

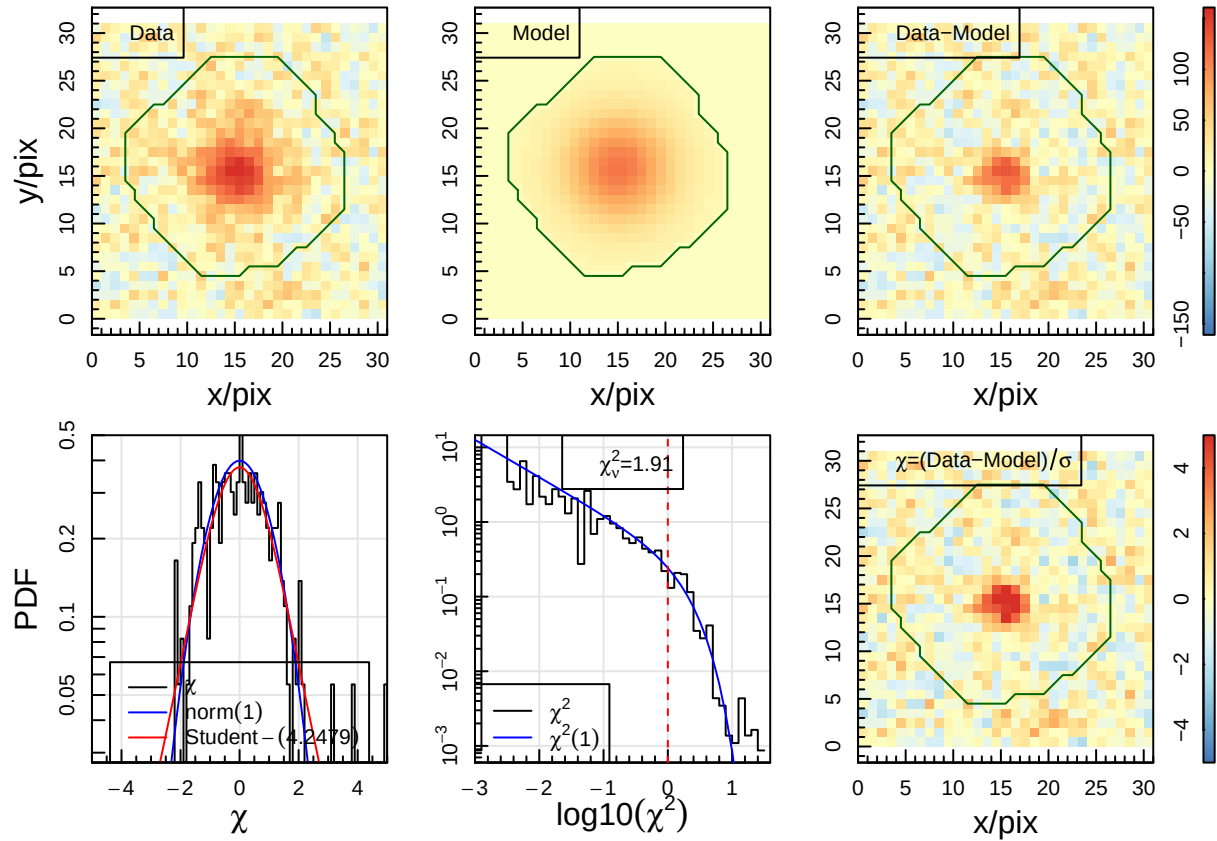
psf_intervals = list(
  moffat = list(
    xcen = list(psf_x+c(-5,5)),
    ycen = list(psf_y+c(-5,5)),
    mag = list(psf_mag+c(-2,2)),
    fwhm = list(c(1,10)),
    con = list(c(1,10)),
    axrat = list(c(0.1,1)),
    box = list(c(-1,1))
  )
)
```

We now set up our model for fitting the star:

```
psf_Data = profitSetupData(psf_image$image, sigma=psf_sigma$image, segim=psf_segim$image,
                           modellist=psf_modellist, tofit=psf_tofit, tolog=psf_tolog,
                           magzero=30, algo.func='optim', intervals=psf_intervals)
```

We can check our initial guess easily:

```
profitLikeModel(parm=psf_Data$init, Data=psf_Data, makeplots=TRUE, plotchisq=TRUE)
```



```
## [1] -681.7616
```

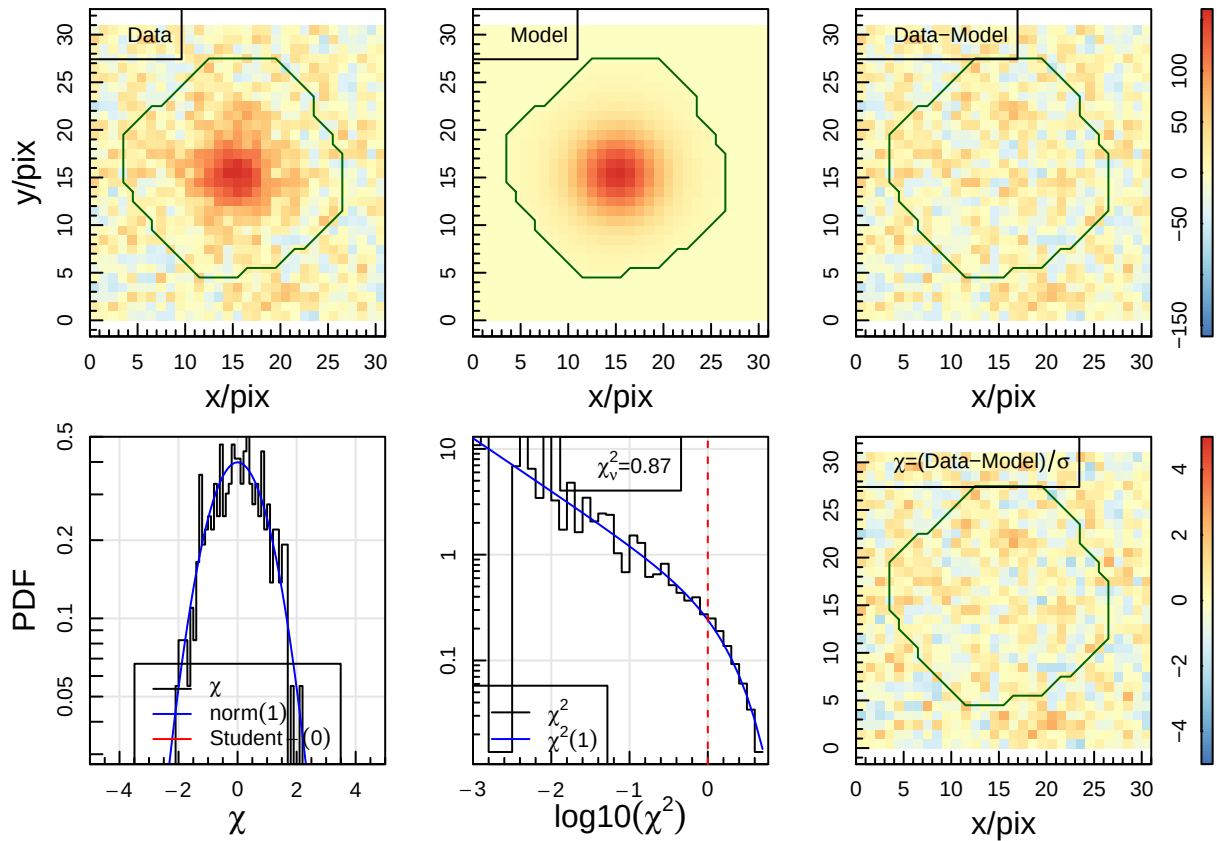
We can do an **optim** fit to improve things:

```
psf_fit = optim(psf_Data$init, profitLikeModel, method='BFGS', Data=psf_Data, control=list(fnscale=-
```

Now we plot the output:

```
profitLikeModel(parm=psf_fit$par, Data=psf_Data, makeplots=TRUE, plotchisq=TRUE)
```

```
## Warning in dt(x, tdof): NaNs produced
```



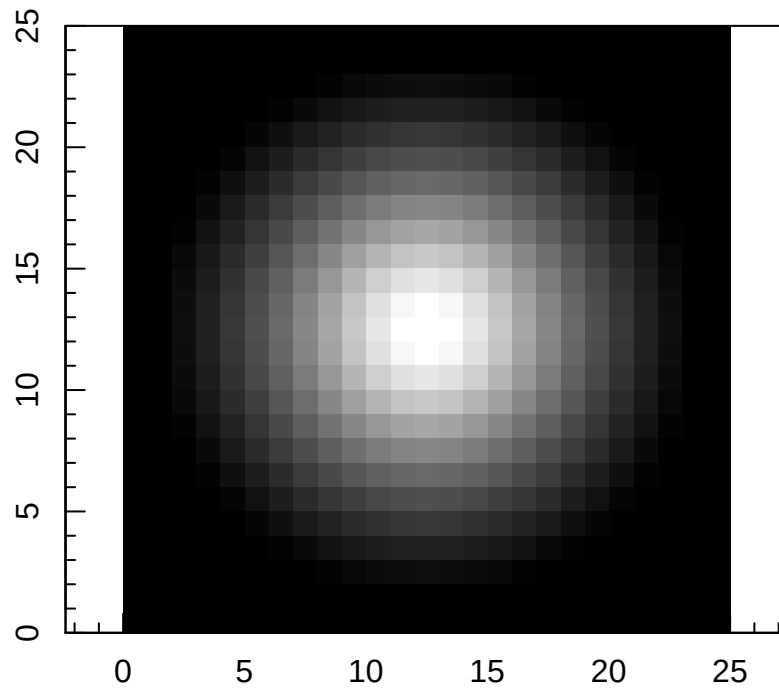
```
## [1] -492.7897
```

This looks like a very good fit. We will now construct our model PSF:

```
psf_modellist_fit = profitRemakeModellist(parm=psf_fit$par, Data=psf_Data)$modellist
psf_modellist_fit$moffat$xcen = 25/2
psf_modellist_fit$moffat$ycen = 25/2
psf_model = profitMakeModel(modellist=psf_modellist_fit, dim=c(25,25))$z
```

And now we can look at our final PSF model:

```
magimage(psf_model)
```



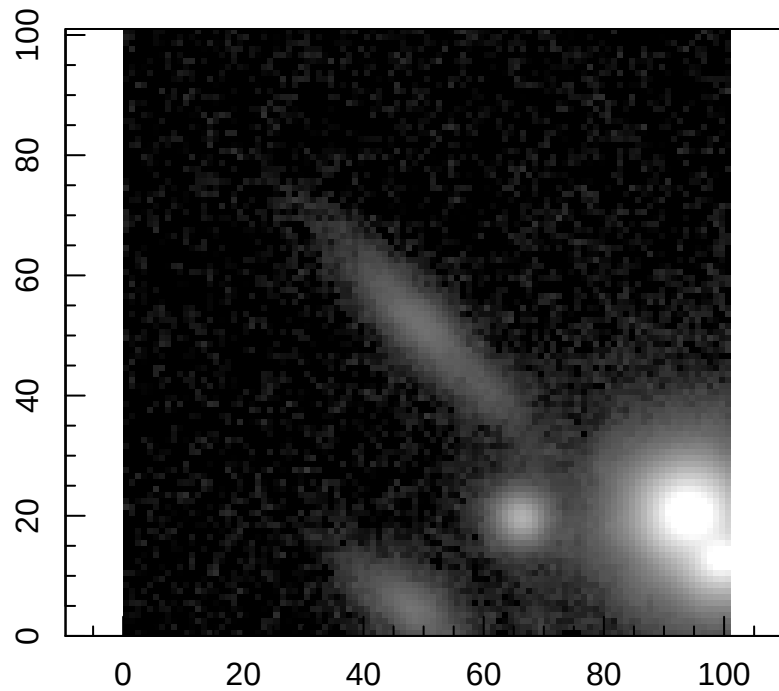
Now we have all the key bits we need in order to fit our target galaxy!

Fitting the Target Galaxy with ProFit

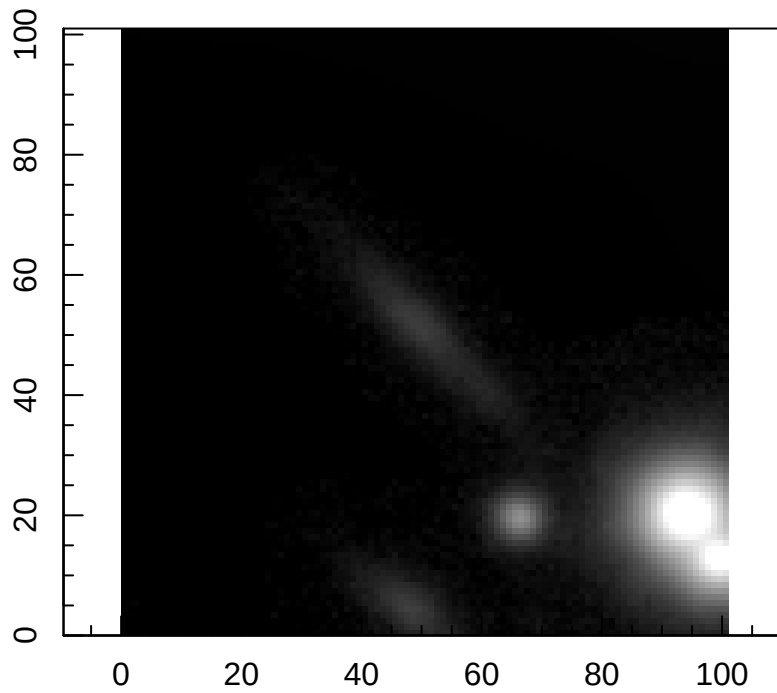
```
gal_image = magcutout(image$imDat, loc=segstats[6,list(xcen,ycen)], box=c(101,101))
gal_sigma = magcutout(sigma, loc=segstats[6,list(xcen,ycen)], box=c(101,101))
gal_segim = magcutout(profound$segim, loc=segstats[6,list(xcen,ycen)], box=c(101,101))
```

Check the cut-out regions:

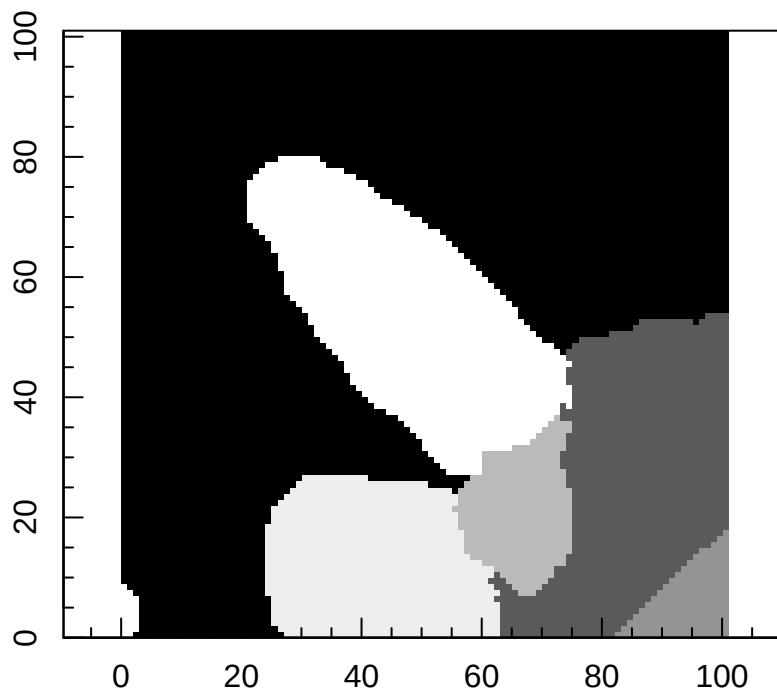
```
magimage(gal_image$image)
```



```
magimage(gal_sigma$image)
```



```
magimage(gal_segim$image)
```



As before, we have to setup some reasonable guesses for our galaxy fit. We give it a very faint initial bulge since we can tell it is a very disk system and we are only using a simple optimiser so we do not want it to get stuck in an odd local minima with a massive bulge. To do this properly and you should use a better optimiser (e.g. full MCMC or genetic algorithm).

```
gal_x = gal_image$loc[1]
gal_y = gal_image$loc[2]
gal_mag = segstats[6,mag]
gal_re = segstats[6,R50]/0.339
gal_ang = segstats[6,ang]
gal_axrat = segstats[6,axrat]

gal_modelist = list(
```

```

sersic = list(
  xcen = rep(gal_x,2),
  ycen = rep(gal_y,2),
  mag = rep(gal_mag,2)+c(2,0.7),
  re = c(gal_re/4,gal_re),
  nser = c(4,1),
  ang = c(0,gal_ang),
  axrat = c(1,gal_axrat),
  box = rep(0,2)
)
)

```

And now we make the other inputs we need for our fitting **Data** structure:

```

gal_tofit = list(
  sersic = list(
    xcen = c(TRUE,NA), #We fit for xcen and tie the two together
    ycen = c(TRUE,NA), #We fit for ycen and tie the two together
    mag = c(TRUE,TRUE), #Fit for both
    re = c(TRUE,TRUE), #Fit for both
    nser = c(FALSE,FALSE), #Fit for neither
    ang = c(FALSE,TRUE), #Fit for disk
    axrat = c(FALSE,TRUE), #Fit for disk
    box = c(FALSE,FALSE) #Fit for neither
  )
)

gal_tolog = list(
  sersic = list(
    xcen = c(FALSE,FALSE),
    ycen = c(FALSE,FALSE),
    mag = c(FALSE,FALSE),
    re = c(TRUE,TRUE), #re is best fit in log space
    nser = c(TRUE,TRUE), #nser is best fit in log space
    ang = c(FALSE,FALSE),
    axrat = c(TRUE,TRUE), #axrat is best fit in log space
    box = c(FALSE,FALSE)
  )
)

gal_intervals = list(
  sersic = list(
    xcen = list(lim=gal_x+c(-5,5),lim=gal_x+c(-5,5)),
    ycen = list(lim=gal_y+c(-5,5),gal_y+c(-5,5)),
    mag = list(lim=c(10,30),lim=c(10,30)),
    re = list(lim=c(1,5),lim=c(1,100)),
    nser = list(lim=c(0.5,20),lim=c(0.5,20)),
    ang = list(lim=c(-180,360),lim=c(-180,360)),
    axrat = list(lim=c(0.001,1),lim=c(0.001,1)),
    box = list(lim=c(-1,1),lim=c(-1,1))
  )
)

```

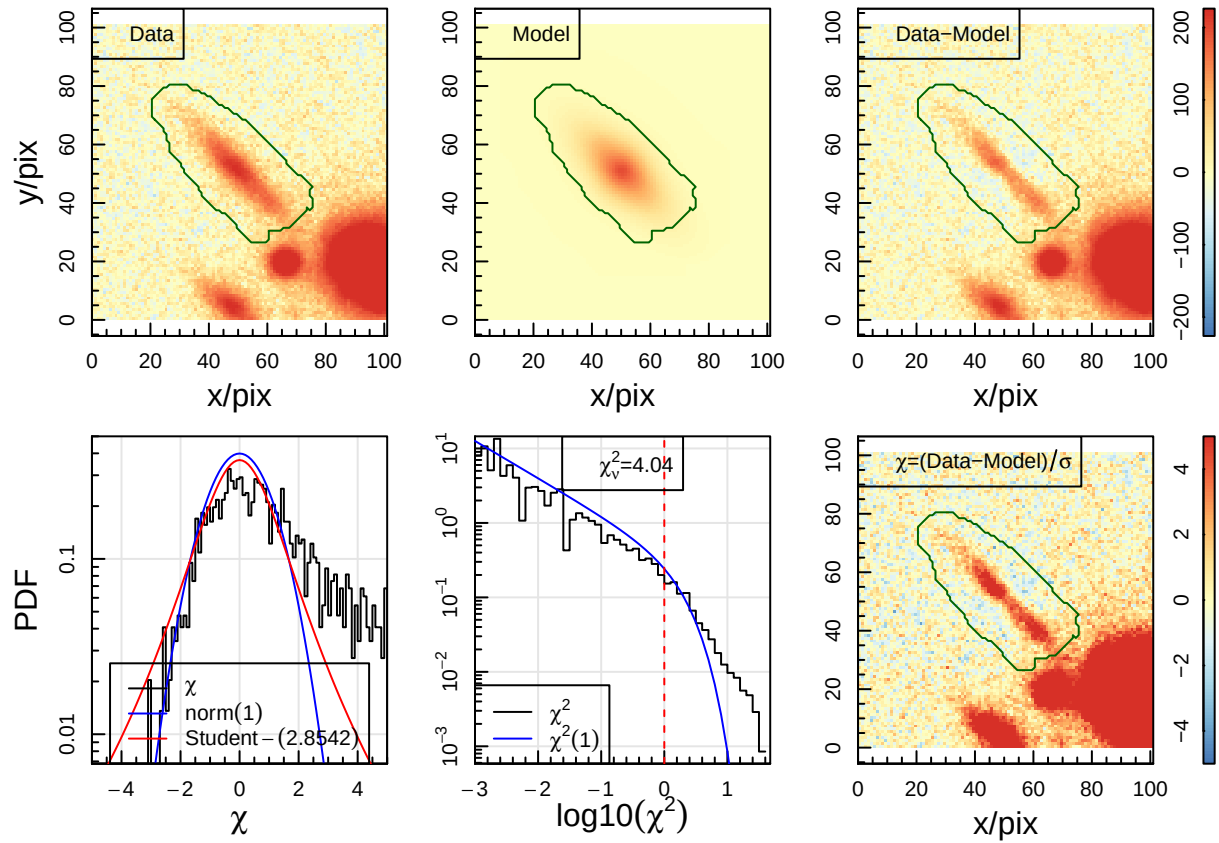
Now we can set up our full fitting structure with everything we have made (notice we use our simpler PSF model here):

```

gal_Data = profitSetupData(gal_image$image, sigma=gal_sigma$image, modellist=gal_modellist,
  tofit=gal_tofit, tolog=gal_tolog, magzero=30, algo.func='optim',
  intervals=gal_intervals, psf=psf_model, segim=gal_segim$image)

```

```
profitLikeModel(parm=gal_Data$init, Data=gal_Data, makeplots=TRUE, plotchisq=TRUE)
```



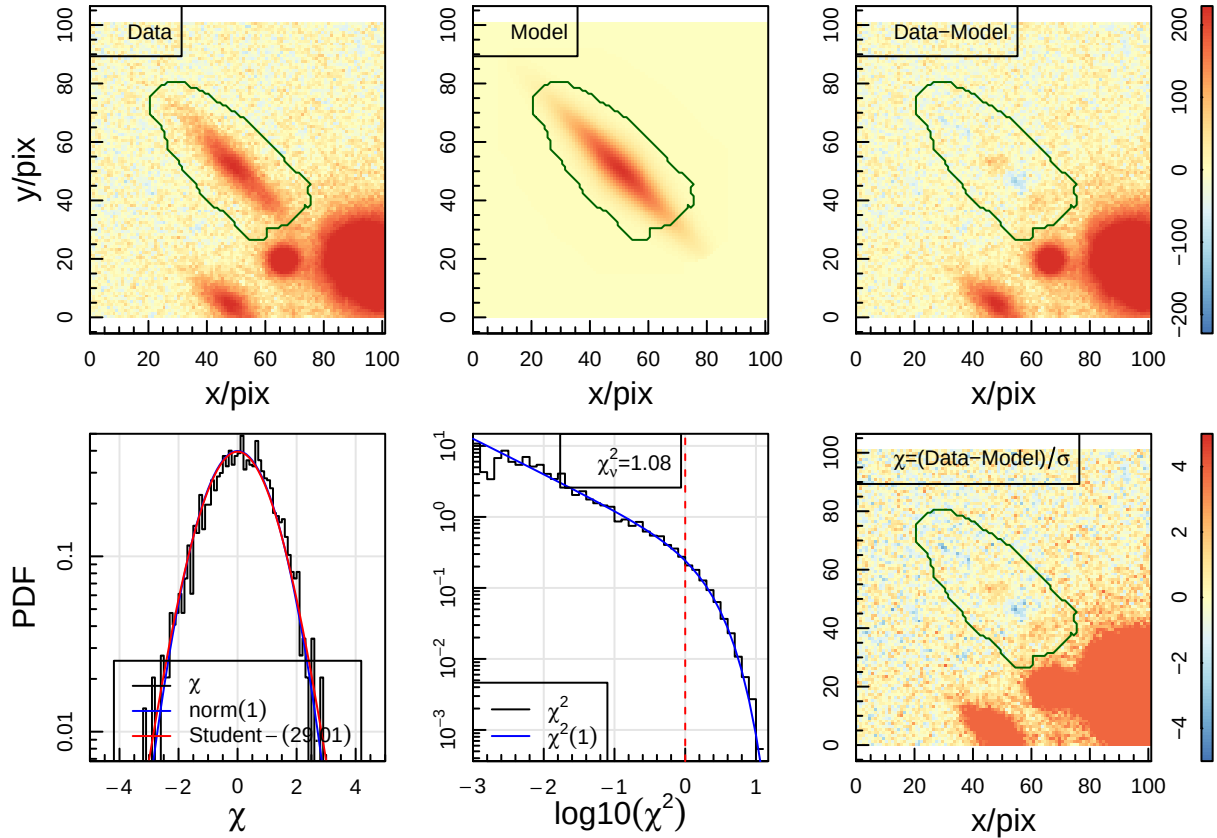
```
## [1] -4327.46
```

Not a bad start, but certainly not great. Let's do an optim fit:

```
gal_fit = optim(gal_Data$init, profitLikeModel, method='BFGS', Data=gal_Data, control=list(fnscale=-
```

We can now check our fit:

```
profitLikeModel(parm=gal_fit$par, Data=gal_Data, makeplots=TRUE, plotchisq=TRUE)
```



```
## [1] -2145.295
```

We can redo this fit using **LaplacesDemon** easily by converting the *algo.func* option to LD:

```
gal_Data$algo.func = 'LD'
```

We can then run **LaplacesDemon** *CHARM* MCMC option via:

```
library(LaplacesDemon)
```

```
##
## Attaching package: 'LaplacesDemon'
##
## The following object is masked from 'package:mixtools':
##
##   ddirichlet
LDout = LaplacesDemon(profitLikeModel, gal_Data, Initial.Values=gal_Data$init, Algorithm='CHARM')
##
## Laplace's Demon was called on Mon Sep 15 17:18:58 2025
##
## Performing initial checks...
## WARNING: Matrix mask may be rank-deficient.
## WARNING: Matrix sigma may be rank-deficient.
## WARNING: Matrix segim may be rank-deficient.
## WARNING: Matrix psf may be rank-deficient.
## WARNING: Matrix region may be rank-deficient.
## WARNING: Matrix calcregion may be rank-deficient.
## WARNING: Matrix model_image_buff may be rank-deficient.
## Suggestion: 18 possible instance(s) of for loops
##   were found in the Model specification. Iteration speed will
##   increase if for loops are vectorized in R or coded in a
##   faster language such as C++ via the Rcpp package.
```



```

## Algorithm: Componentwise Hit-And-Run Metropolis
##
## Laplace's Demon is beginning to update...
## Iteration: 100, Proposal: Componentwise, LP: -2150.3
## Iteration: 200, Proposal: Componentwise, LP: -2144.7
## Iteration: 300, Proposal: Componentwise, LP: -2140.2
## Iteration: 400, Proposal: Componentwise, LP: -2140.6
## Iteration: 500, Proposal: Componentwise, LP: -2141.6
## Iteration: 600, Proposal: Componentwise, LP: -2145.3
## Iteration: 700, Proposal: Componentwise, LP: -2146.2
## Iteration: 800, Proposal: Componentwise, LP: -2144.7
## Iteration: 900, Proposal: Componentwise, LP: -2140.5
## Iteration: 1000, Proposal: Componentwise, LP: -2141.7
## Iteration: 1100, Proposal: Componentwise, LP: -2144
## Iteration: 1200, Proposal: Componentwise, LP: -2140.3
## Iteration: 1300, Proposal: Componentwise, LP: -2142.6
## Iteration: 1400, Proposal: Componentwise, LP: -2144
## Iteration: 1500, Proposal: Componentwise, LP: -2142.2
## Iteration: 1600, Proposal: Componentwise, LP: -2140.2
## Iteration: 1700, Proposal: Componentwise, LP: -2146.6
## Iteration: 1800, Proposal: Componentwise, LP: -2139.8
## Iteration: 1900, Proposal: Componentwise, LP: -2141.4
## Iteration: 2000, Proposal: Componentwise, LP: -2143.3
## Iteration: 2100, Proposal: Componentwise, LP: -2141.6
## Iteration: 2200, Proposal: Componentwise, LP: -2144.4
## Iteration: 2300, Proposal: Componentwise, LP: -2140.3
## Iteration: 2400, Proposal: Componentwise, LP: -2150.5
## Iteration: 2500, Proposal: Componentwise, LP: -2148.4
## Iteration: 2600, Proposal: Componentwise, LP: -2143.3
## Iteration: 2700, Proposal: Componentwise, LP: -2143.2
## Iteration: 2800, Proposal: Componentwise, LP: -2142.9
## Iteration: 2900, Proposal: Componentwise, LP: -2140.3
## Iteration: 3000, Proposal: Componentwise, LP: -2141.4
## Iteration: 3100, Proposal: Componentwise, LP: -2141.3
## Iteration: 3200, Proposal: Componentwise, LP: -2142
## Iteration: 3300, Proposal: Componentwise, LP: -2145.1
## Iteration: 3400, Proposal: Componentwise, LP: -2143.9
## Iteration: 3500, Proposal: Componentwise, LP: -2147.2
## Iteration: 3600, Proposal: Componentwise, LP: -2146.1
## Iteration: 3700, Proposal: Componentwise, LP: -2141.4
## Iteration: 3800, Proposal: Componentwise, LP: -2146.8
## Iteration: 3900, Proposal: Componentwise, LP: -2142.2
## Iteration: 4000, Proposal: Componentwise, LP: -2146.7
## Iteration: 4100, Proposal: Componentwise, LP: -2141.9
## Iteration: 4200, Proposal: Componentwise, LP: -2139.7
## Iteration: 4300, Proposal: Componentwise, LP: -2143.6
## Iteration: 4400, Proposal: Componentwise, LP: -2141
## Iteration: 4500, Proposal: Componentwise, LP: -2144.5
## Iteration: 4600, Proposal: Componentwise, LP: -2144.9
## Iteration: 4700, Proposal: Componentwise, LP: -2141.5
## Iteration: 4800, Proposal: Componentwise, LP: -2144.5
## Iteration: 4900, Proposal: Componentwise, LP: -2143
## Iteration: 5000, Proposal: Componentwise, LP: -2147.8
## Iteration: 5100, Proposal: Componentwise, LP: -2144.3
## Iteration: 5200, Proposal: Componentwise, LP: -2141.6
## Iteration: 5300, Proposal: Componentwise, LP: -2141.9
## Iteration: 5400, Proposal: Componentwise, LP: -2143.7
## Iteration: 5500, Proposal: Componentwise, LP: -2141.6

```

```

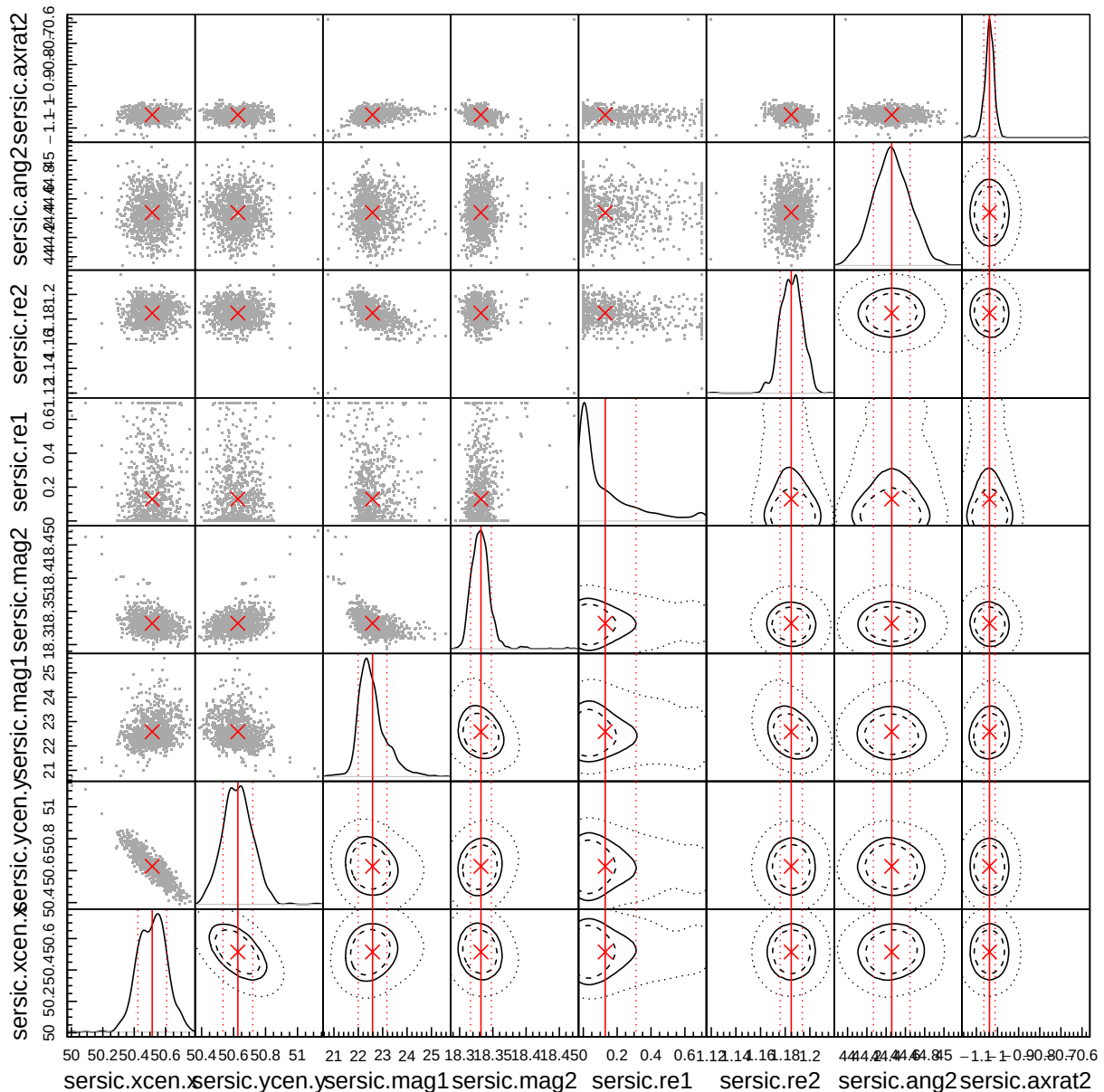
## Iteration: 5600, Proposal: Componentwise, LP: -2141.1
## Iteration: 5700, Proposal: Componentwise, LP: -2144.7
## Iteration: 5800, Proposal: Componentwise, LP: -2144.1
## Iteration: 5900, Proposal: Componentwise, LP: -2143.1
## Iteration: 6000, Proposal: Componentwise, LP: -2141.6
## Iteration: 6100, Proposal: Componentwise, LP: -2142
## Iteration: 6200, Proposal: Componentwise, LP: -2139.4
## Iteration: 6300, Proposal: Componentwise, LP: -2143.3
## Iteration: 6400, Proposal: Componentwise, LP: -2140.1
## Iteration: 6500, Proposal: Componentwise, LP: -2143.2
## Iteration: 6600, Proposal: Componentwise, LP: -2143
## Iteration: 6700, Proposal: Componentwise, LP: -2146.9
## Iteration: 6800, Proposal: Componentwise, LP: -2141.1
## Iteration: 6900, Proposal: Componentwise, LP: -2145.3
## Iteration: 7000, Proposal: Componentwise, LP: -2142.6
## Iteration: 7100, Proposal: Componentwise, LP: -2143.2
## Iteration: 7200, Proposal: Componentwise, LP: -2141.7
## Iteration: 7300, Proposal: Componentwise, LP: -2141.6
## Iteration: 7400, Proposal: Componentwise, LP: -2147.1
## Iteration: 7500, Proposal: Componentwise, LP: -2140.1
## Iteration: 7600, Proposal: Componentwise, LP: -2141
## Iteration: 7700, Proposal: Componentwise, LP: -2140.1
## Iteration: 7800, Proposal: Componentwise, LP: -2144.6
## Iteration: 7900, Proposal: Componentwise, LP: -2143.5
## Iteration: 8000, Proposal: Componentwise, LP: -2141.8
## Iteration: 8100, Proposal: Componentwise, LP: -2145.9
## Iteration: 8200, Proposal: Componentwise, LP: -2139.6
## Iteration: 8300, Proposal: Componentwise, LP: -2140.4
## Iteration: 8400, Proposal: Componentwise, LP: -2140
## Iteration: 8500, Proposal: Componentwise, LP: -2140.2
## Iteration: 8600, Proposal: Componentwise, LP: -2146.5
## Iteration: 8700, Proposal: Componentwise, LP: -2142.2
## Iteration: 8800, Proposal: Componentwise, LP: -2144.6
## Iteration: 8900, Proposal: Componentwise, LP: -2150.3
## Iteration: 9000, Proposal: Componentwise, LP: -2140.4
## Iteration: 9100, Proposal: Componentwise, LP: -2143.8
## Iteration: 9200, Proposal: Componentwise, LP: -2144.9
## Iteration: 9300, Proposal: Componentwise, LP: -2144.6
## Iteration: 9400, Proposal: Componentwise, LP: -2140.3
## Iteration: 9500, Proposal: Componentwise, LP: -2141.1
## Iteration: 9600, Proposal: Componentwise, LP: -2140.5
## Iteration: 9700, Proposal: Componentwise, LP: -2141.5
## Iteration: 9800, Proposal: Componentwise, LP: -2141.8
## Iteration: 9900, Proposal: Componentwise, LP: -2144.1
## Iteration: 10000, Proposal: Componentwise, LP: -2141.5
##
## Assessing Stationarity
## Assessing Thinning and ESS
## Creating Summaries
## Estimating Log of the Marginal Likelihood
## Creating Output
##
## Laplace's Demon has finished.

```

Have a look at the various outputs you get from running this. It will take about 10 minutes to run.

An interesting thing to explore is always the corner/triangle plot, which lets us see how parameters behave with respect to each other when sampling:

```
magtri(LDout$Posterior2)
```



Given the above work flow, the keen student should now consider how to fit the extended source below this one (should be uniqueID=175133, x=175, y=133).

Even Easier

Since mid-2021 a more automated extension of **ProFit** exists called **ProFuse** (available on GitHub at [asgr/ProFuse](https://github.com/asgr/ProFuse) and presented in Robotham 2022). This basically automates all the steps above- taking the image, creating the PSF model based on the likely stars, isolating your object, generating a sensible model without default parameters and fitting range etc, and finally fitting it (this will take about 5 minutes to run locally):

```
library(ProFuse)
lazy = profuseDoFit(image=image$imDat, magzero=30)
```

```
## Running Found2Fit
```

```
## Need PSF for Ncomp >= 1 or Ncomp == 0. Running AllStarDoFit!
```

```

## Running Found2Fit
##      Running ProFound
## Running Highlander
## Iteration 1
## CMA 1: -1600.277 25.5 75.964 25.415 77.15 25.132 25.382 76.583 77.017 20.264 20.706 20.862 20.99
##
## Laplace's Demon was called on Mon Sep 15 17:29:05 2025
##
## Performing initial checks...
## WARNING: Matrix mask may be rank-deficient.
## WARNING: Matrix sigma may be rank-deficient.
## WARNING: Matrix segim may be rank-deficient.
## WARNING: Matrix region may be rank-deficient.
## WARNING: Matrix calcregion may be rank-deficient.
## WARNING: Matrix model_image_buff may be rank-deficient.
## Algorithm: Componentwise Hit-And-Run Metropolis
##
## Laplace's Demon is beginning to update...
## Iteration: 100, Proposal: Componentwise, LP: -1589.8
##
## Assessing Stationarity
## Assessing Thinning and ESS
## Creating Summaries
## Creating Output
##
## Laplace's Demon has finished.
## LD Mode 1: -1588.683 25.372 76.067 25.452 76.98 25.057 25.39 76.635 76.838 20.282 20.629 20.831 2
## Iteration 2
##
## Laplace's Demon was called on Mon Sep 15 17:29:09 2025
##
## Performing initial checks...
## WARNING: Matrix mask may be rank-deficient.
## WARNING: Matrix sigma may be rank-deficient.
## WARNING: Matrix segim may be rank-deficient.
## WARNING: Matrix region may be rank-deficient.
## WARNING: Matrix calcregion may be rank-deficient.
## WARNING: Matrix model_image_buff may be rank-deficient.
## Algorithm: Componentwise Hit-And-Run Metropolis
##
## Laplace's Demon is beginning to update...
## Iteration: 100, Proposal: Componentwise, LP: -1592.5
##
## Assessing Stationarity
## Assessing Thinning and ESS
## Creating Summaries
## Creating Output
##
## Laplace's Demon has finished.
## LD Mode 2: -1586.182 25.446 76.187 25.494 77.098 25.008 25.406 76.647 76.757 20.304 20.665 20.861
##      Running ProFound
## Running Highlander

```

```

## Iteration 1
## CMA 1: -2097.421 30.574 23.56 20.126 18.386 1.71 1.177 44.47 -0.981
##
## Laplace's Demon was called on Mon Sep 15 17:29:35 2025
##
## Performing initial checks...
## WARNING: Matrix mask may be rank-deficient.
## WARNING: Matrix segim may be rank-deficient.
## WARNING: Matrix psf may be rank-deficient.
## WARNING: Matrix region may be rank-deficient.
## WARNING: Matrix calcregion may be rank-deficient.
## WARNING: Matrix model_image_buff may be rank-deficient.
## Algorithm: Componentwise Hit-And-Run Metropolis
##
## Laplace's Demon is beginning to update...
## Iteration: 100, Proposal: Componentwise, LP: -2103.3
## Iteration: 200, Proposal: Componentwise, LP: -2104.4
##
## Assessing Stationarity
## Assessing Thinning and ESS
## Creating Summaries
## Creating Output
##
## Laplace's Demon has finished.

## Iteration 2
##
## Laplace's Demon was called on Mon Sep 15 17:30:19 2025
##
## Performing initial checks...
## WARNING: Matrix mask may be rank-deficient.
## WARNING: Matrix segim may be rank-deficient.
## WARNING: Matrix psf may be rank-deficient.
## WARNING: Matrix region may be rank-deficient.
## WARNING: Matrix calcregion may be rank-deficient.
## WARNING: Matrix model_image_buff may be rank-deficient.
## Algorithm: Componentwise Hit-And-Run Metropolis
##
## Laplace's Demon is beginning to update...
## Iteration: 100, Proposal: Componentwise, LP: -2107.3
## Iteration: 200, Proposal: Componentwise, LP: -2101.4
##
## Assessing Stationarity
## Assessing Thinning and ESS
## Creating Summaries
## Creating Output
##
## Laplace's Demon has finished.

## LD Mode 2: -2097.421 30.574 23.56 20.126 18.386 1.711 1.177 44.47 -0.981

## Iteration 3
##
## Laplace's Demon was called on Mon Sep 15 17:31:03 2025
##
## Performing initial checks...
## WARNING: Matrix mask may be rank-deficient.
## WARNING: Matrix segim may be rank-deficient.

```

```

## WARNING: Matrix psf may be rank-deficient.
## WARNING: Matrix region may be rank-deficient.
## WARNING: Matrix calcregion may be rank-deficient.
## WARNING: Matrix model_image_buff may be rank-deficient.
## Algorithm: Componentwise Hit-And-Run Metropolis
##
## Laplace's Demon is beginning to update...
## Iteration: 100, Proposal: Componentwise, LP: -2098.7
## Iteration: 200, Proposal: Componentwise, LP: -2099.8
##
## Assessing Stationarity
## Assessing Thinning and ESS
## Creating Summaries
## Creating Output
##
## Laplace's Demon has finished.

## Iteration 4

##
## Laplace's Demon was called on Mon Sep 15 17:31:48 2025
##
## Performing initial checks...
## WARNING: Matrix mask may be rank-deficient.
## WARNING: Matrix segim may be rank-deficient.
## WARNING: Matrix psf may be rank-deficient.
## WARNING: Matrix region may be rank-deficient.
## WARNING: Matrix calcregion may be rank-deficient.
## WARNING: Matrix model_image_buff may be rank-deficient.
## Algorithm: Componentwise Hit-And-Run Metropolis
##
## Laplace's Demon is beginning to update...
## Iteration: 100, Proposal: Componentwise, LP: -2100.2
## Iteration: 200, Proposal: Componentwise, LP: -2099.7
##
## Assessing Stationarity
## Assessing Thinning and ESS
## Creating Summaries
## Creating Output
##
## Laplace's Demon has finished.

## Iteration 5

##
## Laplace's Demon was called on Mon Sep 15 17:32:32 2025
##
## Performing initial checks...
## WARNING: Matrix mask may be rank-deficient.
## WARNING: Matrix segim may be rank-deficient.
## WARNING: Matrix psf may be rank-deficient.
## WARNING: Matrix region may be rank-deficient.
## WARNING: Matrix calcregion may be rank-deficient.
## WARNING: Matrix model_image_buff may be rank-deficient.
## Algorithm: Componentwise Hit-And-Run Metropolis
##
## Laplace's Demon is beginning to update...
## Iteration: 100, Proposal: Componentwise, LP: -2099.2
## Iteration: 200, Proposal: Componentwise, LP: -2100.4
## Iteration: 300, Proposal: Componentwise, LP: -2102

```

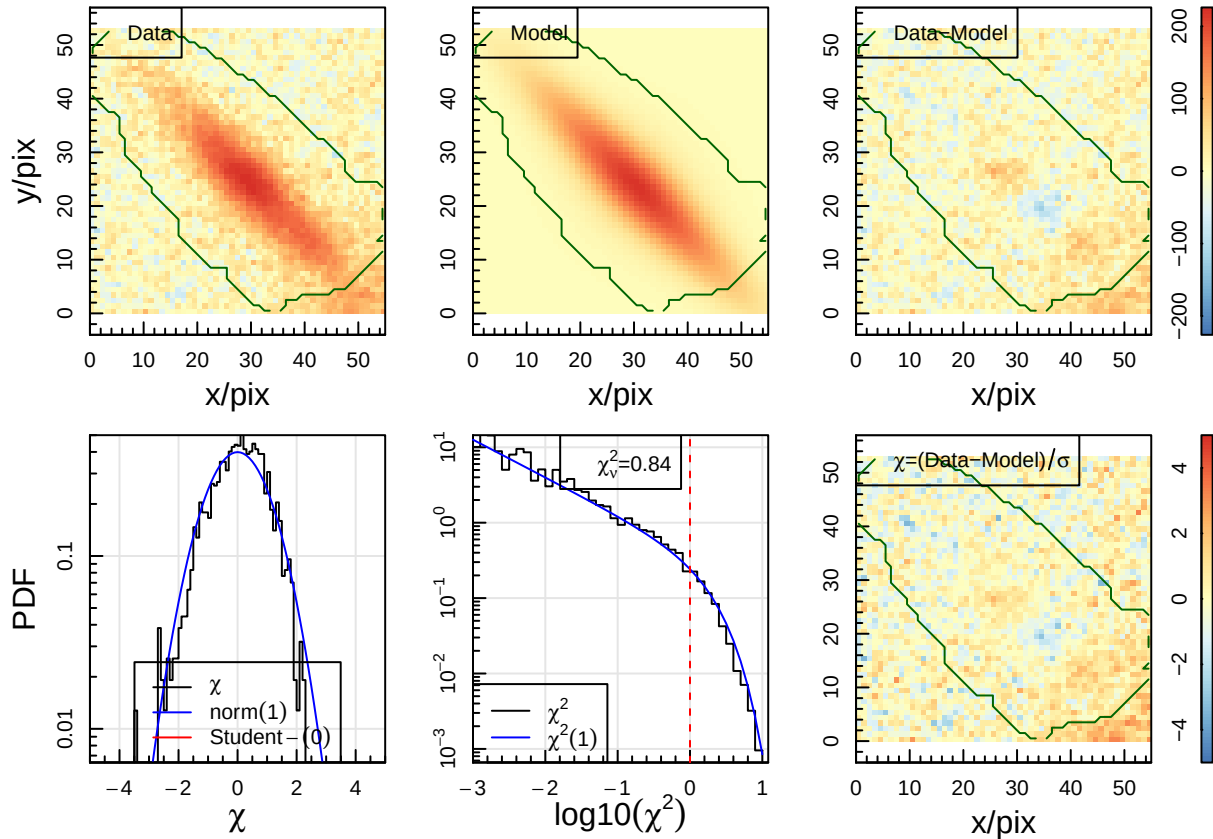
```
## Iteration: 400, Proposal: Componentwise, LP: -2103.6
## Iteration: 500, Proposal: Componentwise, LP: -2103.4
## Iteration: 600, Proposal: Componentwise, LP: -2100.8
## Iteration: 700, Proposal: Componentwise, LP: -2102.7
## Iteration: 800, Proposal: Componentwise, LP: -2102.7
## Iteration: 900, Proposal: Componentwise, LP: -2103
## Iteration: 1000, Proposal: Componentwise, LP: -2109.8
##
## Assessing Stationarity
## Assessing Thinning and ESS
## Creating Summaries
## Creating Output
##
## Laplace's Demon has finished.
```

That was easy! There are complex types of fits that you might need to do more manually with **ProFit**, but the above appears to cover 99% of fitting situations in practice.

We can have a look at the results in a similar manner to before:

```
profitLikeModel(lazy$parm, Data=lazy$Data, makeplots=TRUE, plotchisq=TRUE)
```

```
## Warning in dt(x, tdof): NaNs produced
```



```
## $LP
## [1] -2097.421
##
## $Dev
## [1] 4194.842
##
## $Monitor
##      LL      LP tend.elapsed
## -2097.421 -2097.421      940.618
```

```
##
## $yhat
## [1] 1
##
## $parm
## sersic.xcen1 sersic.ycen1 sersic.mag1 sersic.mag2 sersic.re1
## 30.5735883 23.5599684 20.1263921 18.3859817 1.7109431
## sersic.re2 sersic.ang2 sersic.axrat2
## 1.1774566 44.4704656 -0.9807026
```

Explore the output of *lazy* and the various helper functions to figure out how to present the posterior samples of the last MCMC output.